

| Practical Messaging — Day Two

A 101 guide to messaging. Ian Cooper. (X, BlueSky and Hachyderm: ICooper)

Why Event-Driven?

Two slides. Day 1 taught the mechanics; this says what they were for, and hands into the design day.

Easy to Change, and Robust

There are many drivers that cause us to distribute a system — **performance and scalability, availability, maintainability**, and applications that are **inherently distributed**. Strip the classification buckets away and the two properties you are actually buying are:

- **Easy to change** — ship a part without shipping the whole. *Independent deployability*. One team decides its own release candidate and is in production in hours, not at the end of a two-week release everyone had to agree a date for.
- **Robust** — keep working when something you depend on is not. *Guaranteed delivery*.

Both are bought with the same mechanism: **messages** — which is what yesterday was about.

Two properties, one mechanism. Independent deployability is the prize; everything else is about not giving it back.

So How Do You Design One?

Yesterday answered *how do I send and receive reliably*. Today asks the harder question:

- **What shape is the system**, once no one call-and-returns its way through the whole use case?
- **Who is in charge** — and does anyone need to be?
- **Where does the work live** when a request is not a thread waiting for a reply?

Temporal coupling multiplies outages. Yesterday settled what to do about it.

Messaging doesn't remove the outage; it converts a failure into a delay. Now: what does a system built that way actually look like?

Flow and Reactive Programming

The opening section of Day Two, in four movements — and the question under all of them is what is in charge.

Flow and Reactive Programming

| Everything in this section is one question: *what is in charge?*

Four movements:

- **How the office did it** — paper workflows, and the failures they already had answers for.
- **How would you build that?** — call and return, at object scale and at service scale.
- **The formalism** — dataflow, then flow-based programming: the same flows as a graph.
- **The name** — and you have been building it since yesterday morning.

How the Office Did It

Paper workflows: the frame, the desk, the worked flows and how they failed — closing with the hotel's, drawn by you.

Paper Workflows

| "My life looked good on paper — where, in fact, almost all of it was being lived." —
Martin Amis

Before computers, offices ran large distributed systems on paper. They handled concurrency, failure, recovery and scale — and there was **no main**. Nobody held the whole process. Every desk did one thing, and it did it because something arrived in its in-tray.

| The office was a distributed system with no god object. It worked for two hundred years.

The Frame

In a mail room, **the frame** was the rack of pigeonholes: mail was sorted into it by floor. **A mail cart** then picked mail up from the frame and distributed it.

- The frame creates **channels** you post to.
- Contents are delivered to whoever subscribes to that pigeonhole.
- The sender does not wait, and does not know who collects.
- Nobody walks to the frame to look. **The cart is what moves it**, desk by desk.

If the frame is the broker, with its topics, then the cart is the queue, distributing the messages to consumers. We built both yesterday.



Skinny and Fat, on Paper

Two things arrive in an in-tray, and you met them both yesterday.

- An **interdepartmental envelope** — one form, one hand-off, one action. That is a **skinny** message.
- A **sack of folders** — the whole file carried over, because the next desk may need any of it. That is a **fat** message.

Day 1 §6.1 named these. The office had both, and it chose per hand-off.

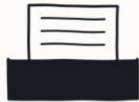


The Desk – In-Tray, Out-Tray, File

every hand-off goes out-tray to in-tray – nobody shouts across the office



*in-tray – work that has arrived,
and is not done yet*



*out-tray – finished here,
and not yet collected*



*the file – what this desk knows,
written down, because the clerk
goes home at five*



a heavy bar – an organisational boundary



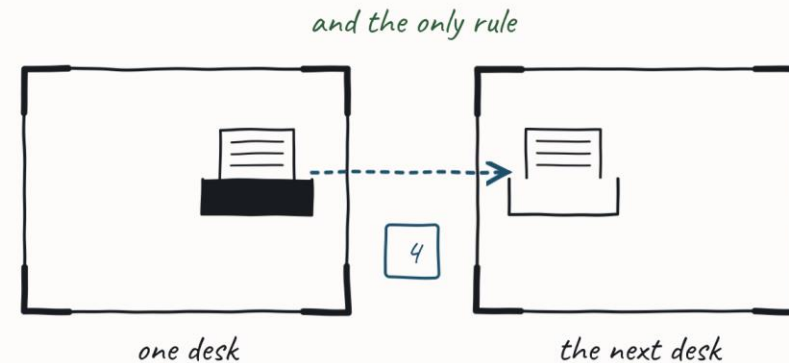
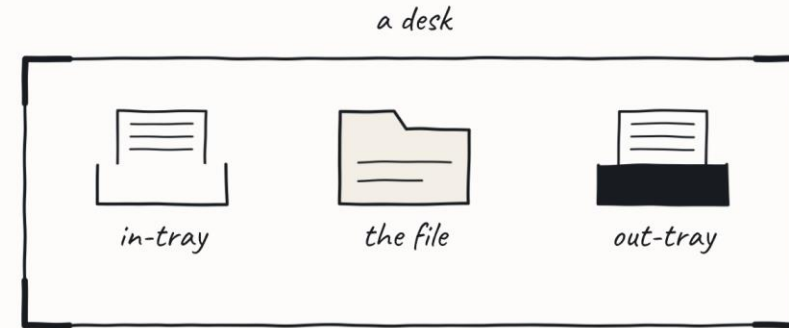
dashed – paper moving



solid – a phone call or a fax



numbered steps show the sequence



Worked Example: Just Paper Takeaway

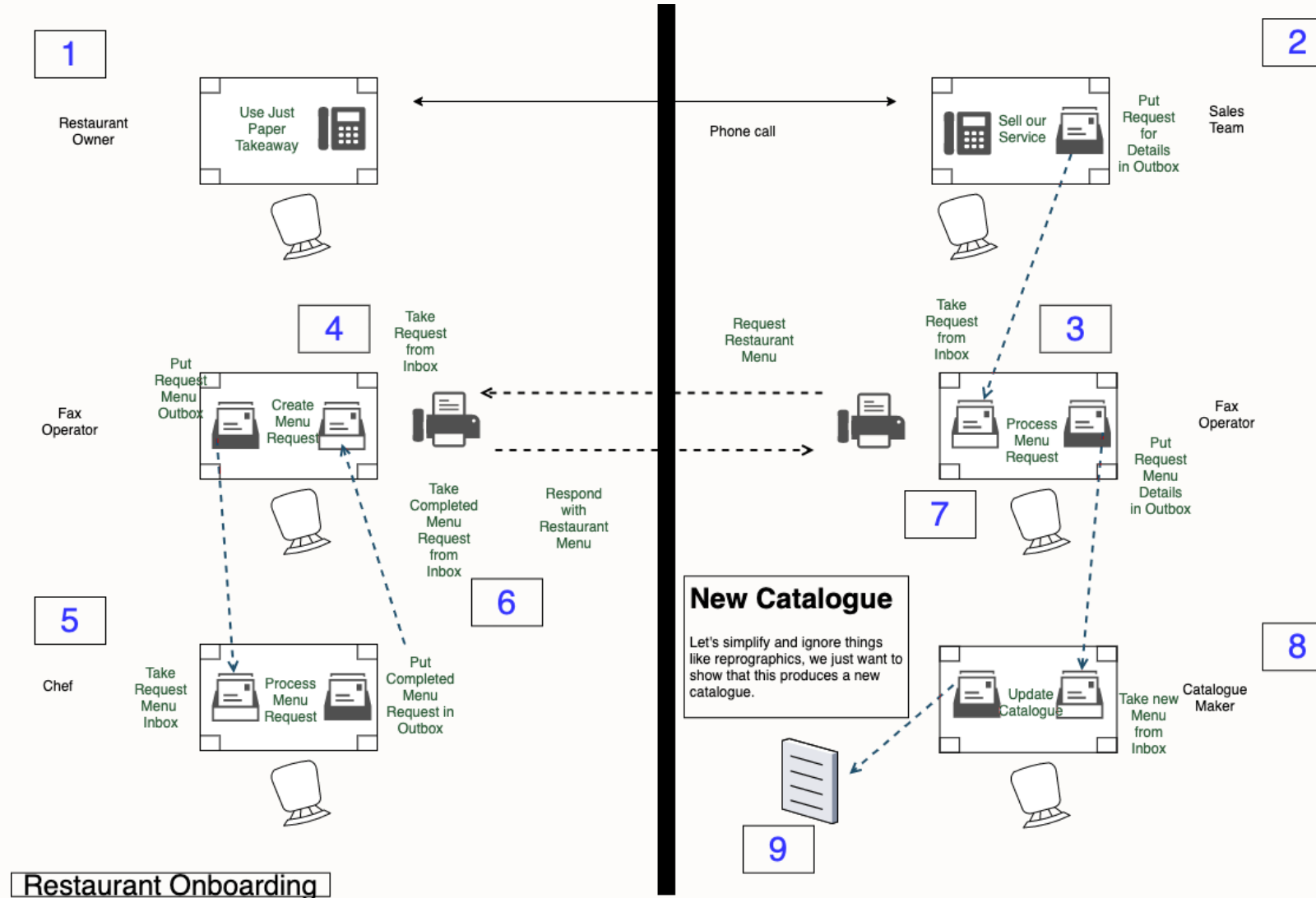
We are going to demonstrate how a paper flow used to work. Our business produces a paper catalogue of local restaurant menus that we mail to nearby households. Customers phone us to order from the catalogue. We then pass the order to the restaurant. Restaurants handle their own delivery.

Worked Flow — Restaurant Onboarding

Signing up a new restaurant, as paper.

- Who the desks are: Restaurant Owner, Sales Team, Fax Operator, Chef, Catalogue Maker.
- Where the organisational boundary falls.
- Every hand-off annotated *Put X in Outbox / Take X from Inbox*.

Worked Flow – Restaurant Onboarding



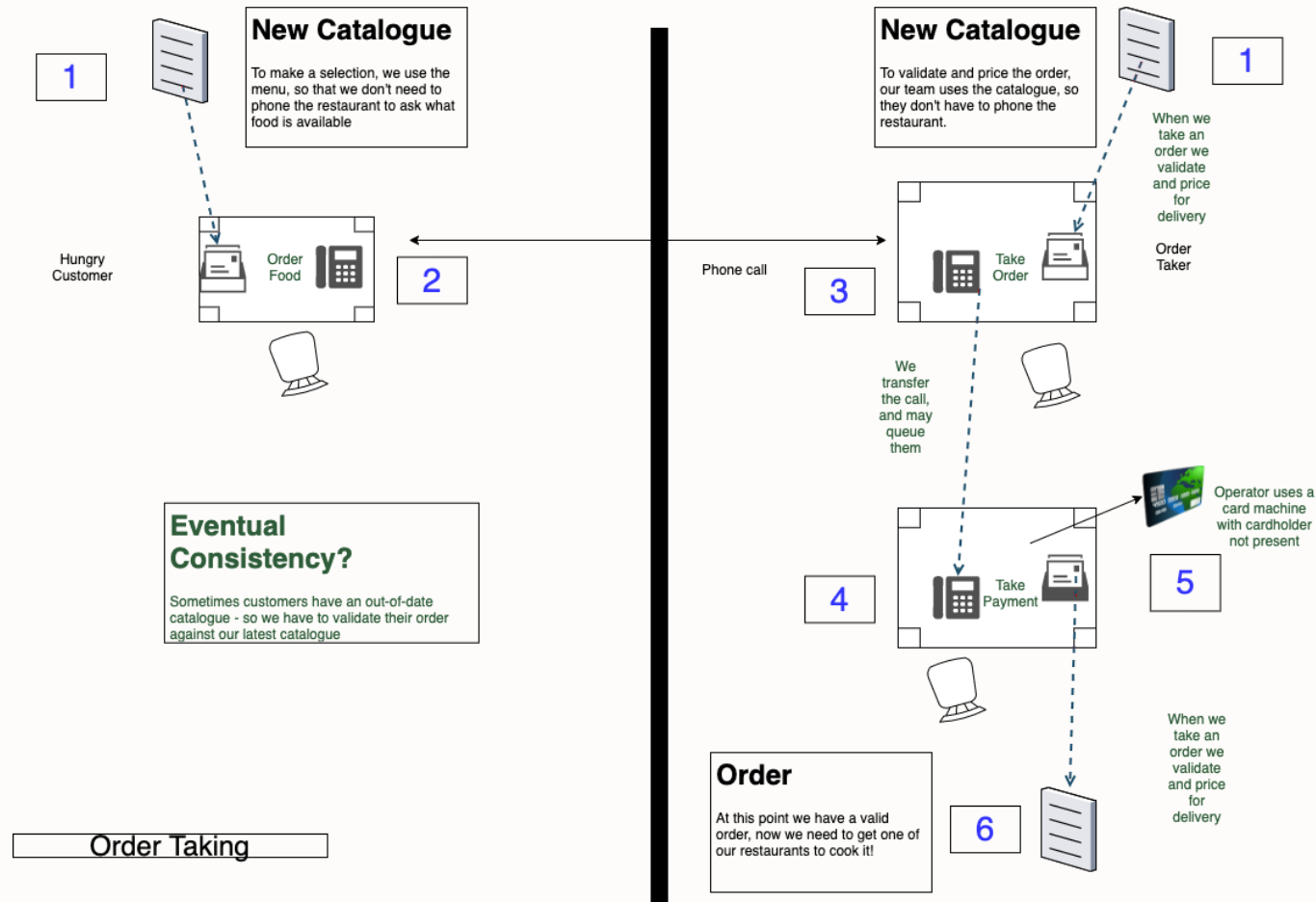
Worked Flow — Customer Order

The customer orders. The first flow that crosses into the customer's world, and the first with more than one channel in it.

- Order taking is **phone, card machine and order pad** — three channels, one flow, and none of them is a computer.
- The order pad is the in-tray; the pad's carbon copy is the outbox; the till roll is the file.

Worked Flow — Customer Order

Three different media, one notation. The notation does not care what the arrow is made of.

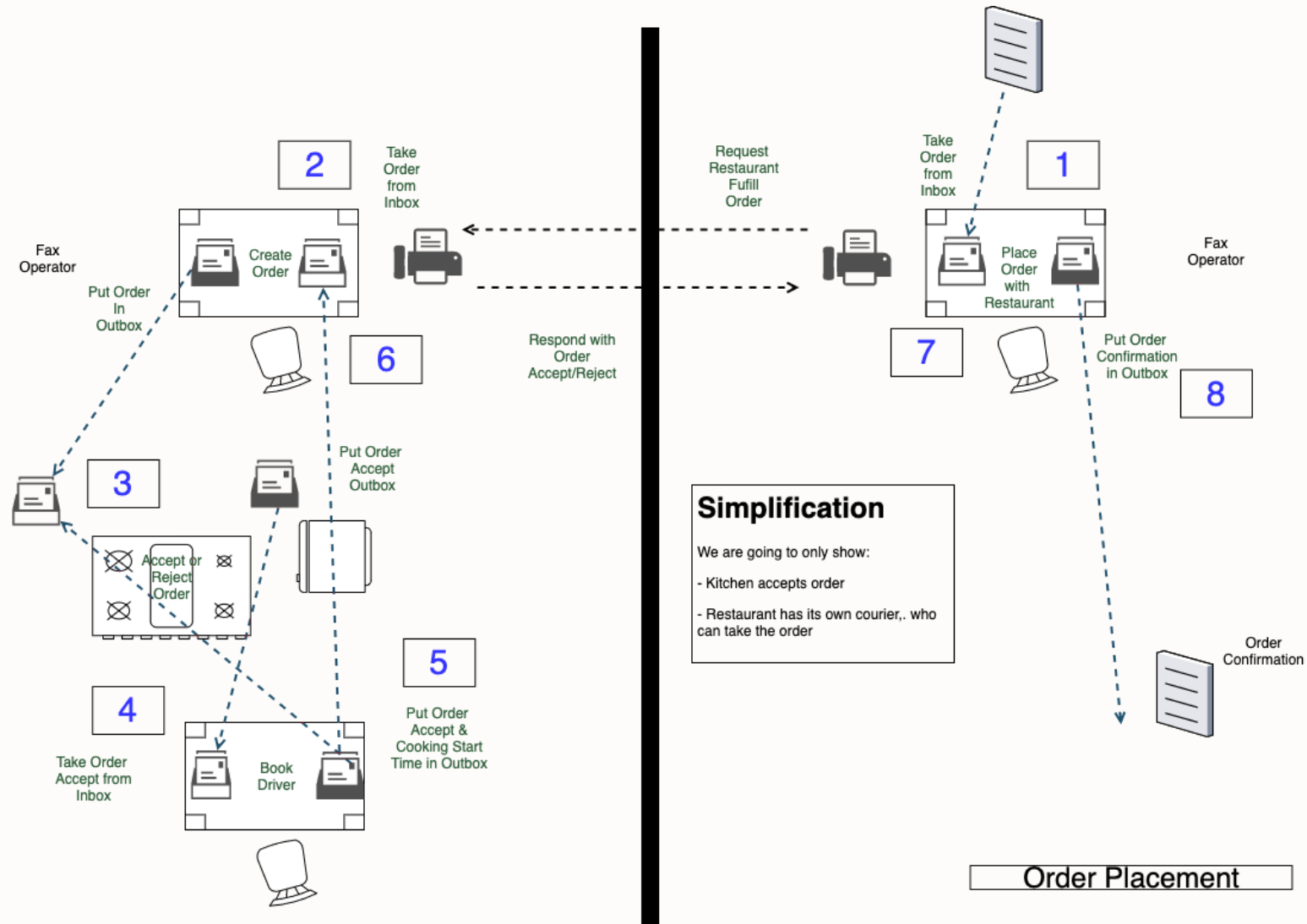


Worked Flow — Order Placement

The order reaches the restaurant. This is the flow with the external participant — the restaurant is another organisation, on the far side of the heavy bar, reachable only by fax.

- The fax is a **send with no immediate answer**. The order goes out; the desk moves on.
- The state has to be **filed** before the fax goes, because the answer arrives minutes later and possibly to a different clerk.
- The reference number written on the fax is what matches the answer back to the file.

Worked Flow – Order Placement

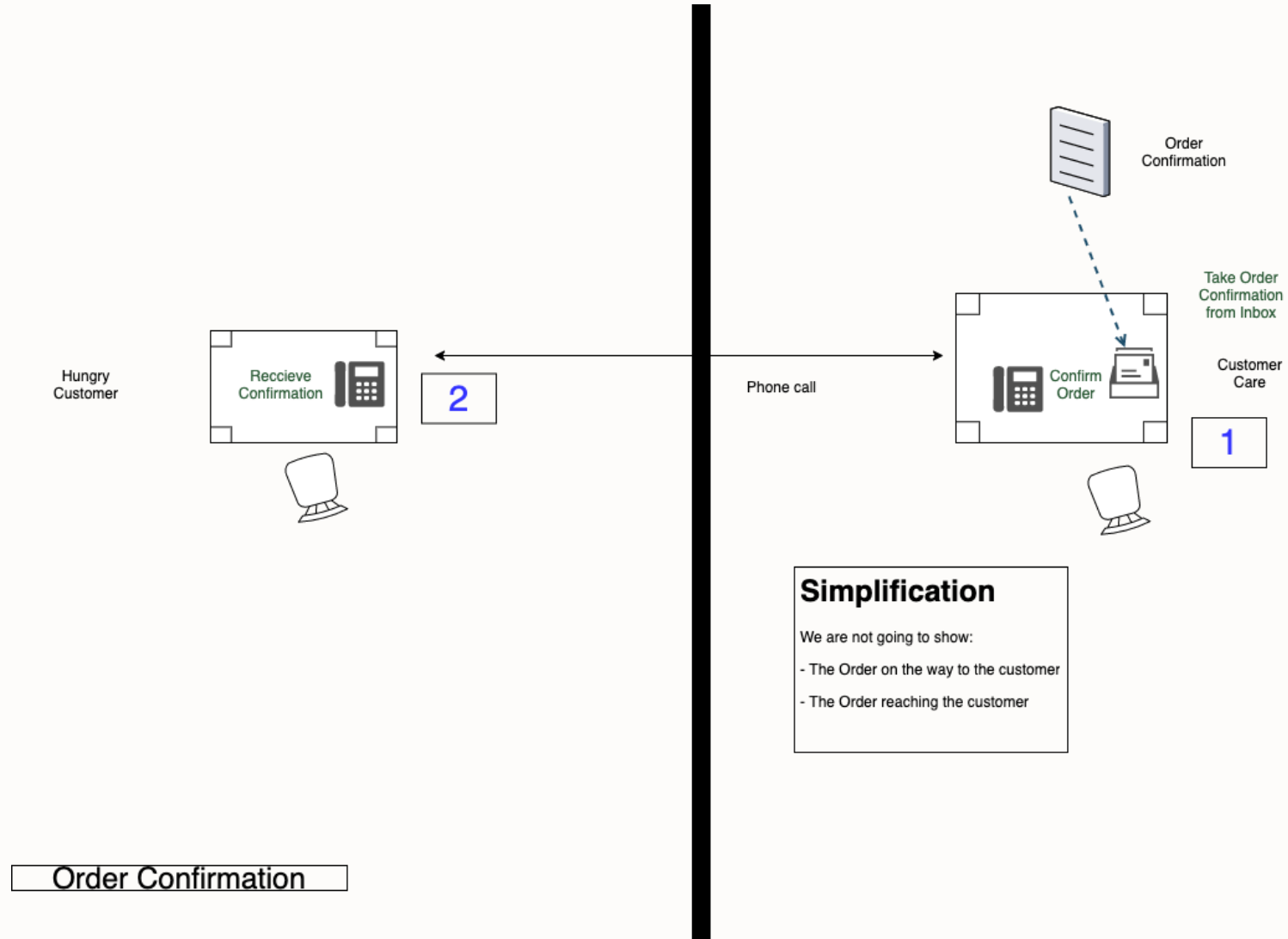


Worked Flow — Order Confirmation

The restaurant confirms back. The reply arrives on its own, later, with no one waiting for it.

- Nobody blocked. The clerk who sent the fax did other work; the confirmation is picked up from an in-tray like anything else.
- The confirmation is matched to the filed order by its reference number, and only then does the customer get told.
- Put the four flows side by side: **no desk knows more than its own step, and there is no coordinator.**

Worked Flow — Order Confirmation



Worked Flow – Order Confirmation

The diagrams illustrate the evolution of a restaurant ordering system through four stages:

- Restaurant Onboarding:** The initial setup where a Restaurant Owner, Chef, and Fax Operator interact with a New Catalogue. The process involves creating a menu, updating it, and taking orders from the inbox.
- Customer Order:** A Hungry Customer places an order via a Phone call. The Order Taker and Operator process the order, validate it, and price it for delivery.
- Order Placement:** A Hungry Customer places an order via a Phone call. The Order Taker and Operator process the order, validate it, and price it for delivery.
- Order Confirmation:** A Hungry Customer places an order via a Phone call. The Order Taker and Operator process the order, validate it, and price it for delivery.

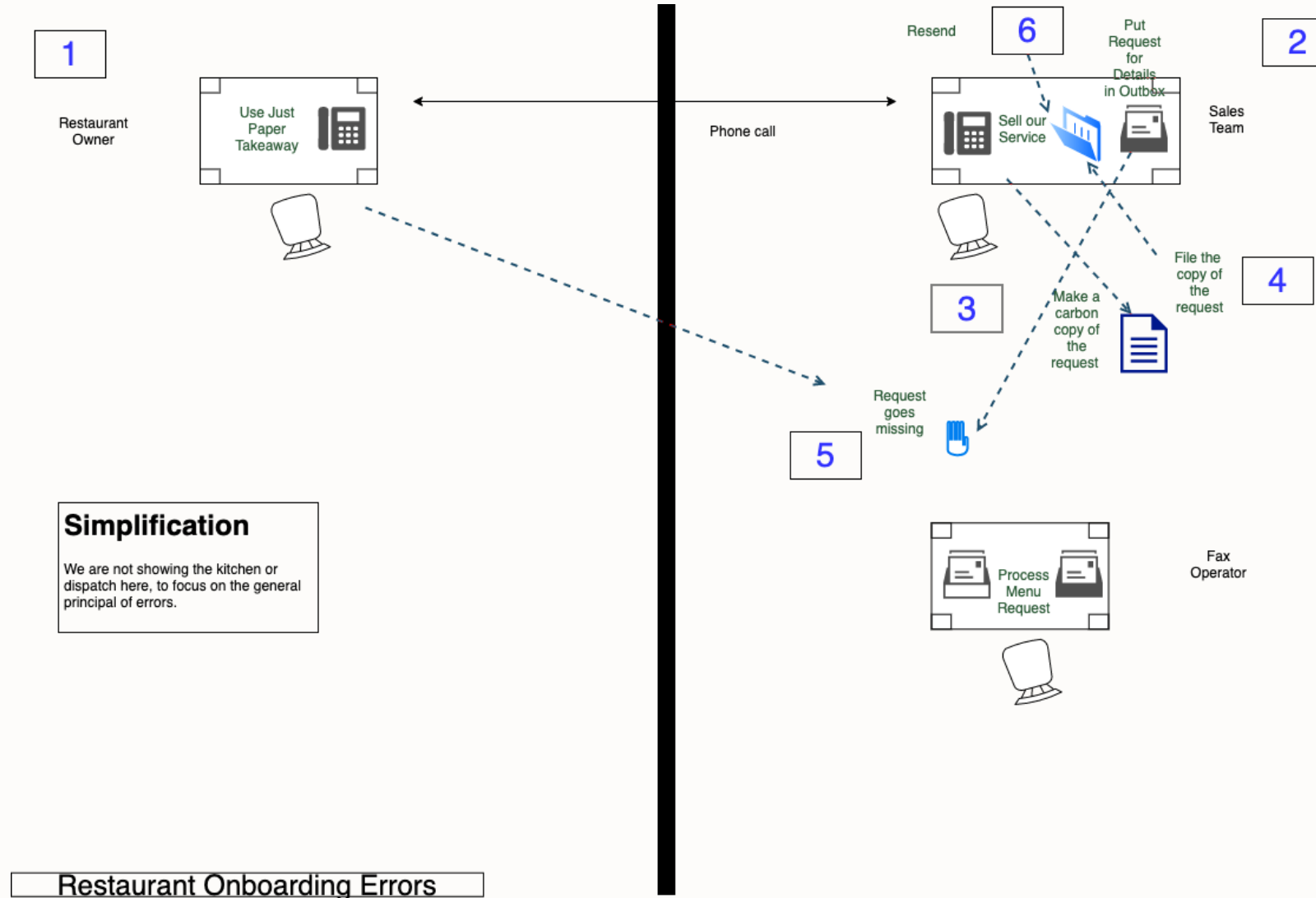
How Do We Deal with Errors?

Paper had failure modes, and it had answers, and they are our answers.

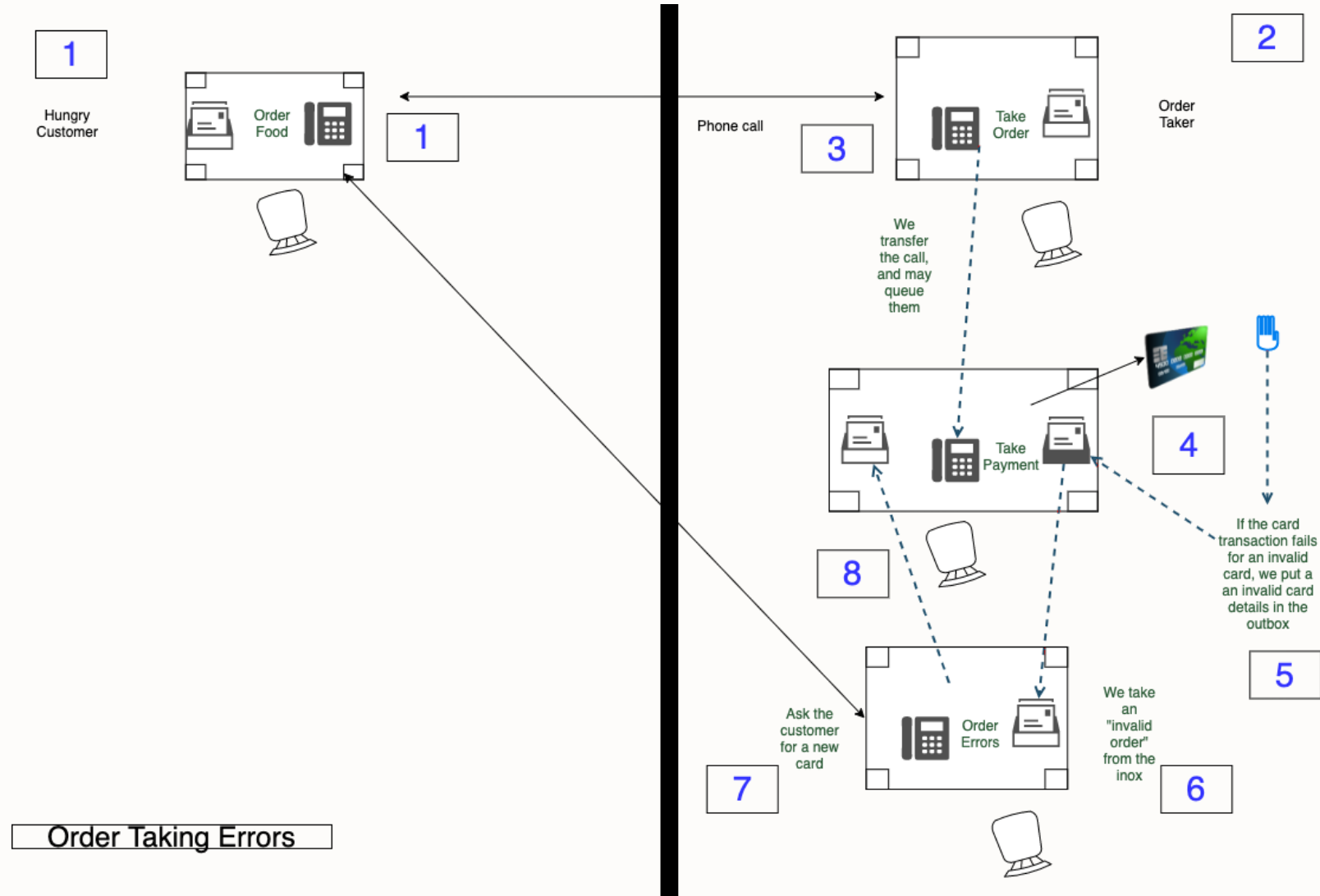
what goes wrong	what the office did	what we call it
the request goes missing	resend the fax	retry, In-Out-Retry
we don't know if it arrived	if no receipt, retry sending	at-least-once
we need to know what we sent	make a carbon copy before you send	the outbox
we need it after we go home	file the copy	durable state
it arrived twice	check the file before acting	idempotency, the inbox

Fax Call Log								
Monday, 2010-11-08 11:19								
Date	Time	Type	Job #	Length	Speed	Station Name/Number	Pages	Status
2010-06-18	08:31	SCAN	92	0:25	28800	[REDACTED]	0	E-705 V.34 1M31
2010-03-22	11:39	RECV	59	0:20	26400	[REDACTED]	1	OK -- V.34 BM31
2010-03-22	11:45	RECV	60	0:20	26400	[REDACTED]	1	OK -- V.34 BM31
2010-03-22	12:29	RECV	61	0:21	26400	[REDACTED]	1	OK -- V.34 BM31
2010-09-14	14:46	SCAN	129	1:40	9600	[REDACTED]	2	E-606 V.29 AR30

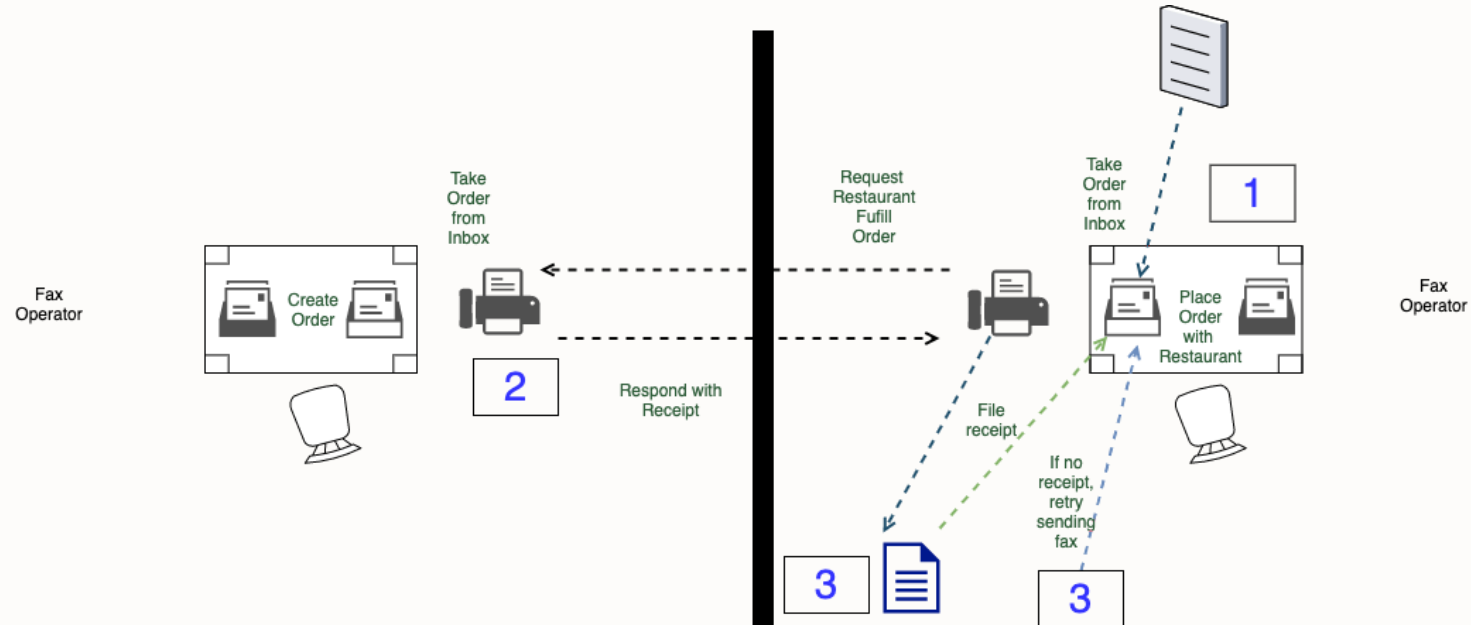
How Do We Deal with Errors?



How Do We Deal with Errors?



How Do We Deal with Errors?



Simplification

We are not showing the kitchen or dispatch here, to focus on the general principal of errors.

Order Placement Errors

Now Do One — the Hotel, on Paper

You have seen the takeaway's four flows, and you have seen them fail.

Now draw the hotel's.

- One stage of the guest cycle per table — Onboarding, Pre-Arrival, Arrival, Occupancy, Departure.
- **Hard rule: every hand-off goes out-tray to in-tray.** Nobody shouts across the office.
- Then run it with cards, one person per desk. Then we break it.



DON'T PANIC

How Would You Build That?

Call and return, at object scale and at service scale — ending in the distributed monolith, which is the wrong answer, offered second.

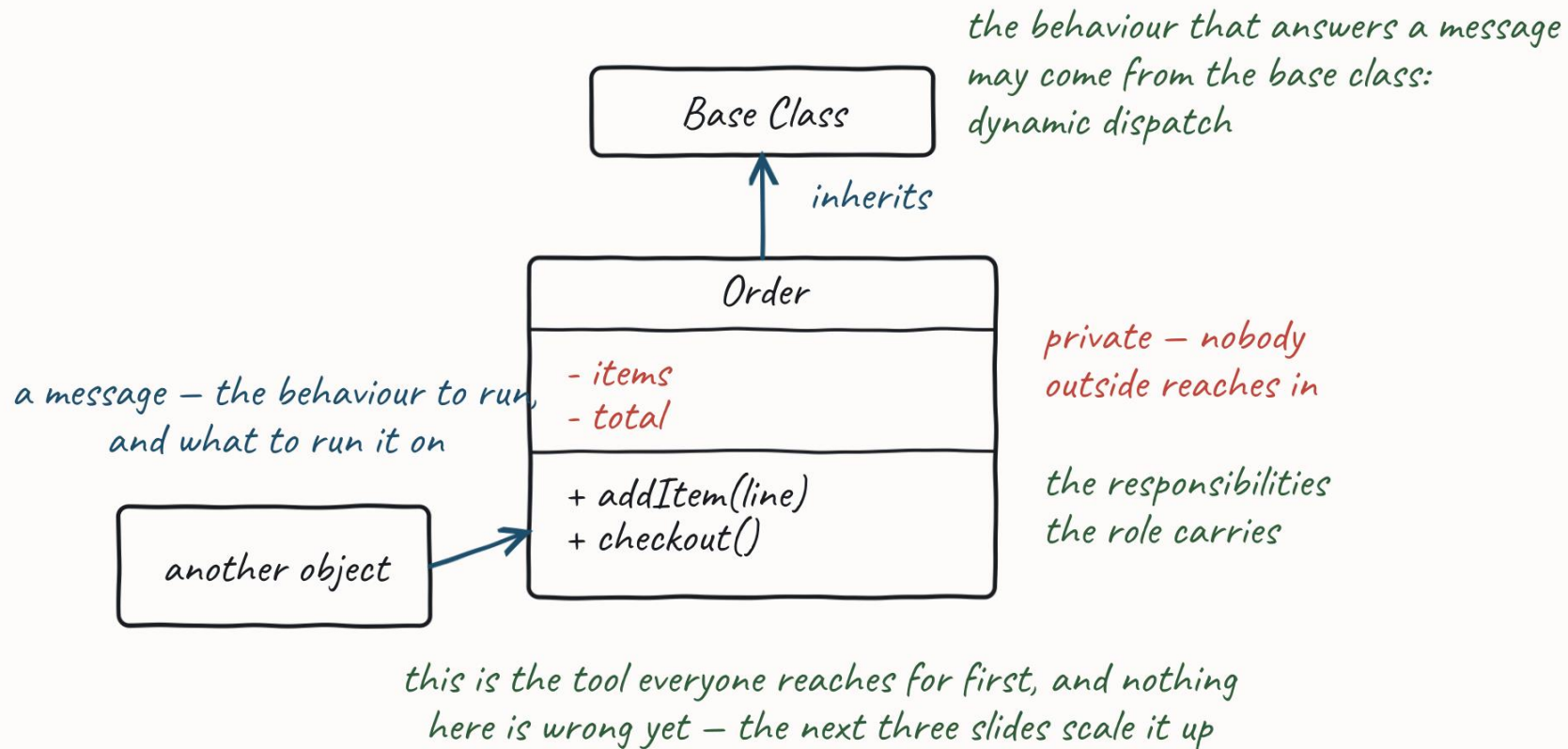
Object-Oriented Programming

You have now drawn a flow twice — once watching, once yourselves. Asked to *build* it, almost everyone reaches for the same tool first. It is still on the flipchart from before the exercise, so let us name it properly before we test it.

A class has a **role** with **responsibilities**; we capture responsibilities as behaviours; we encapsulate the data those behaviours need inside the object; roles may be inherited via dynamic dispatch.

Object-Oriented Programming

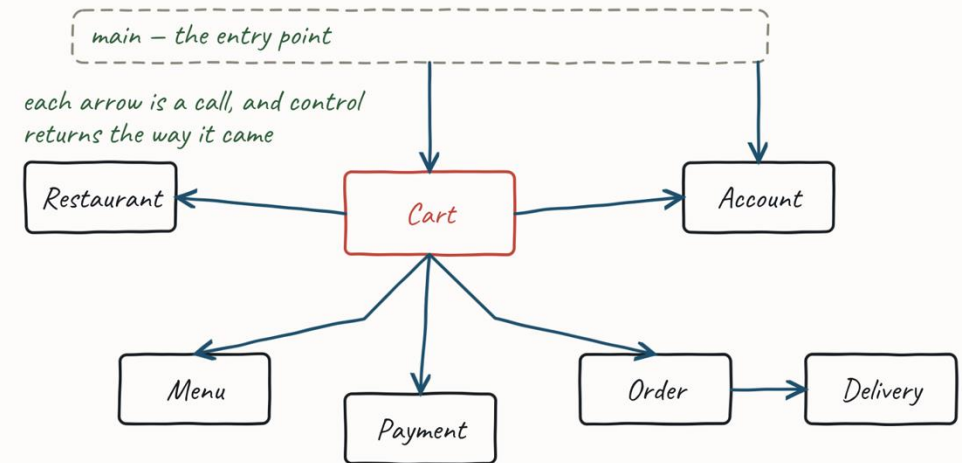
the data lives inside the object, and only the object's own behaviours may touch it



Call and Return, and the God Object

- **Call and return.** `main` is the entry point. It invokes objects, which invoke other objects and return to their caller. Control is passed down a stack and handed back.
- **The god object.** The danger is one object — `Cart`, usually — that controls all the others. High behavioural coupling: it knows the whole use case, so it changes whenever any step of the use case changes.

every step of the use case passes through one object — so it changes whenever any step does



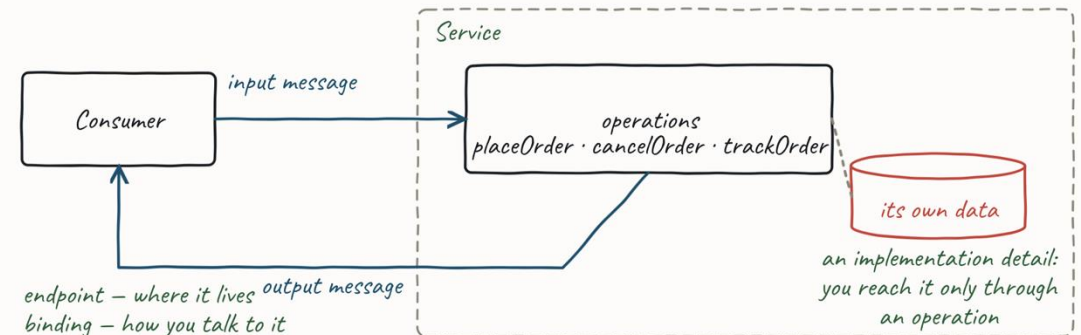
no desk in this morning's takeaway knew the whole process. `Cart` knows all of it.

SOA Is OO at Macro Scale

SOA creates OO-like components: a service has a role and responsibilities exposed as operations, and encapsulates the data those operations need. This is the Web Services approach — OO as an architectural principle.

- Same idea, bigger unit: role, responsibilities, encapsulated data.
- We are taking the view that **microservices are SOA 3.0** — most of the best practice still applies.

same idea, a bigger unit: operations, and the data those operations need, shut away behind them

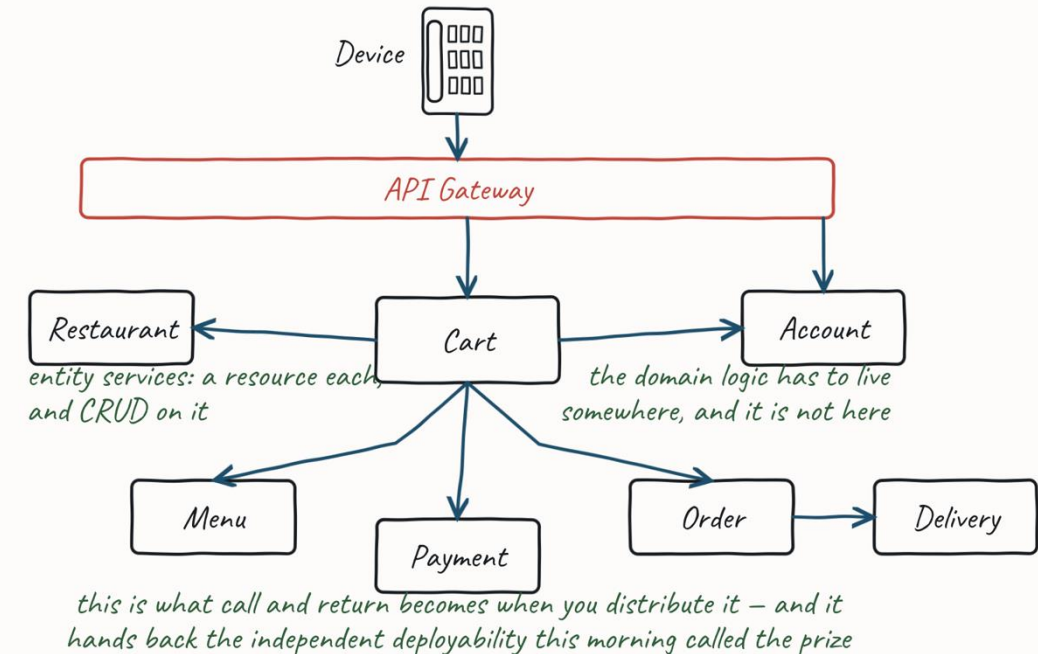


“a service should represent a self-contained functionality that corresponds to a real-world business activity” – and a desk is a business activity

Feature Envy – You Built a Distributed Monolith

- We expose significant **resources** and the operations you can perform on them — usually CRUD. Entity services: Cart, Restaurant, Account, Menu, Payment, Order, Delivery.
- **Feature envy.** Domain logic has to coordinate across those resources, so it ends up on the client, in the API gateway, or in the Cart service — anywhere but in the individual services.
- Because we made OO large, we reached for call and return via a main method at the API gateway.

you distributed the objects and kept the god object — the gateway is 'main'



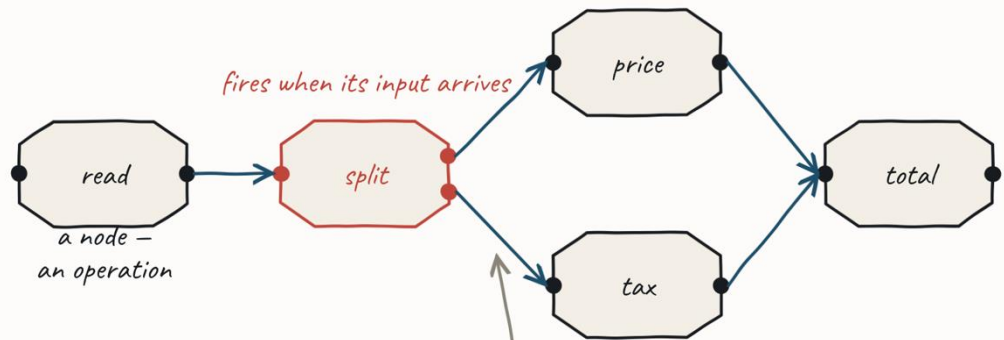
The Formalism

Dataflow, then flow-based programming — the same flows again as a graph, closing with yours redrawn.

Data Flow Programming

Dataflow programming conceptualises a program as a directed graph: operations are **nodes**, connected by **arcs** through which data flows. A node performs its operation when its input data is available — not when someone calls it.

- Nothing is in charge. There is no main.
- You already use one: the Unix command line. Pipes and filters.

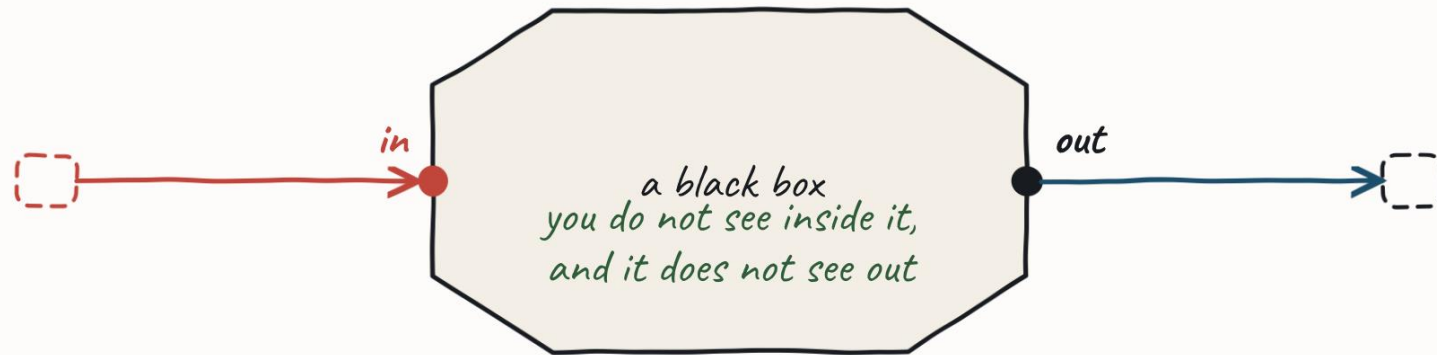


a node runs because a packet arrived on its input — nothing calls it, and there is no 'main'

*an arc — where the data moves
in call and return, control moves and data sits still. Here it is
the other way round — and you already use one: pipes and filters.*

Nodes, Ports and Firing

it is reactive because it fires in response to an event – a packet arriving is the event



activation – it fires

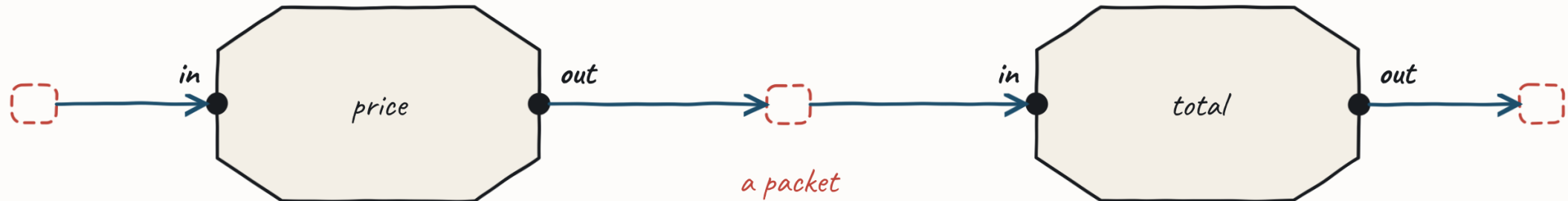
process, and make an answer

push the answer out

*generally a node is single-threaded: one clerk does one document at a time.
Throughput comes from having many nodes, not from one node doing many things.*

Nodes, Ports and Firing

the only thing that crosses between two nodes is data – no call, no return, no stack

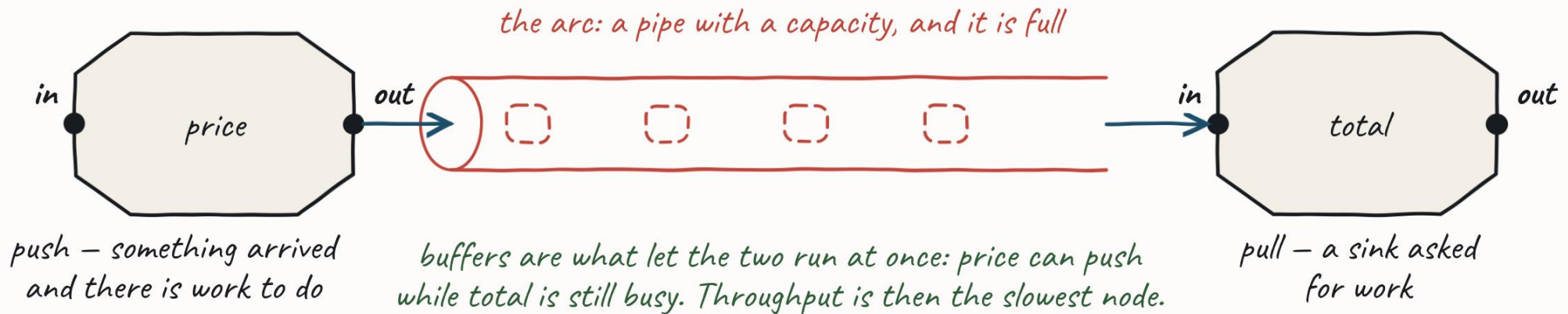


*just data: a primitive or a compound value, and it may
carry other packets inside it*

neither node knows who wrote the packet or who will read the next one – it knows the name of a port

Capacity, Backpressure and Node Lifetime

we do not fire a node to read its input if there is no space on its output – a full buffer is the whole of backpressure



links with infinite capacity exist only in theory, so when the buffer fills there are exactly two answers: slow the producer, or drop data

Flow-Based Programming

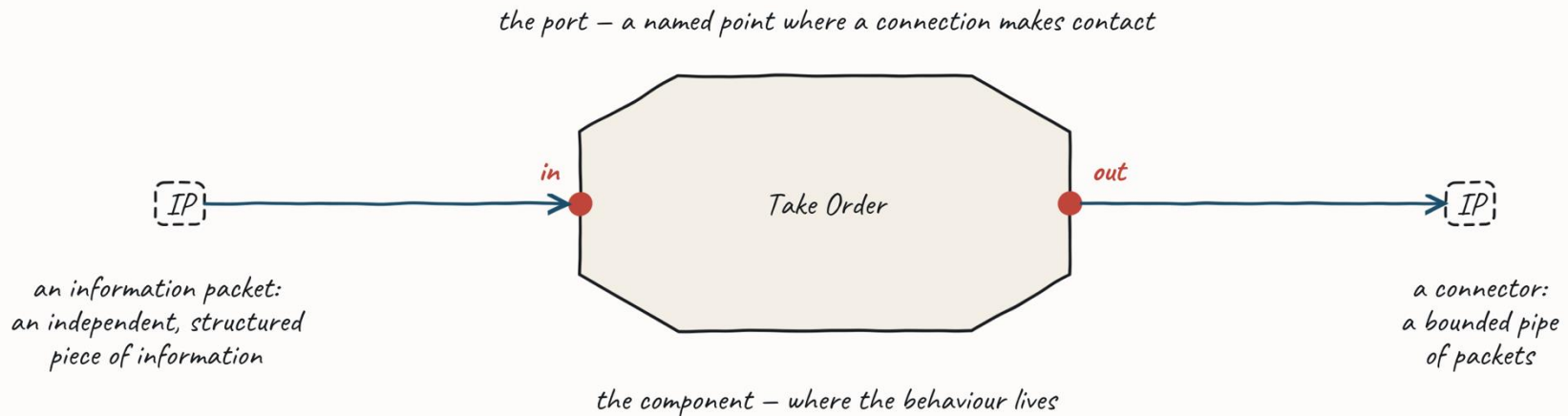
Flow-Based Programming (FBP) is a subclass of dataflow. Where dataflow *may* be synchronous, FBP always is not:

- **Always asynchronous.**
- **Multiple input ports** per component.
- **Bounded buffers**, and therefore backpressure when they fill. A component can test whether it can send to out, and decide whether to halt or ignore.
- **Node lifetime:** a component keeps running while there is work on an input queue, and **suspends** rather than terminates when there is none.

Flow-Based Programming

| "Everything flows and nothing stays." — Heraclitus

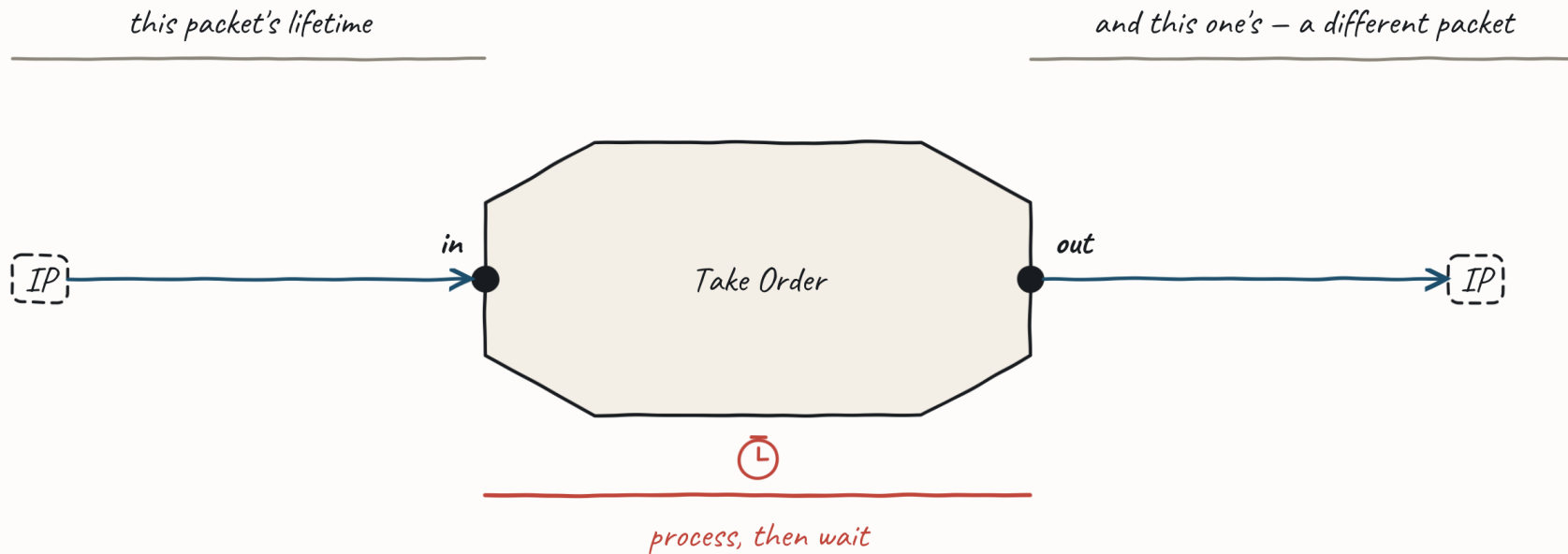
a component knows the name of a port and nothing about what is behind it – so the connector can be swapped for anything



in OO the object holds the state; in FBP the state is in the packet, and it is out there on a connector rather than in storage

Flow-Based Programming

a component suspends when there is no work – it does not terminate, and it is not started per packet

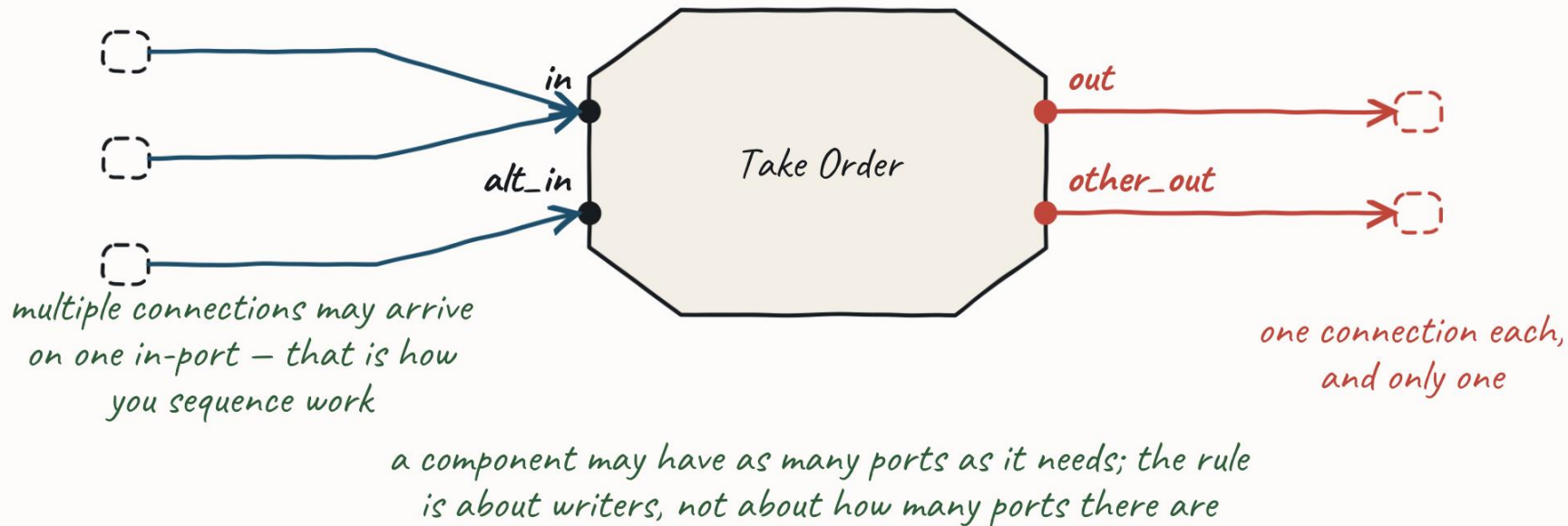


consuming a packet destroys it; sending one creates a new one. The component outlives both.

this is the message pump from Day 1, described from the other side of the loop

Flow-Based Programming

many writers may share an in-port; an out-port has exactly one – so what leaves a component is never contended

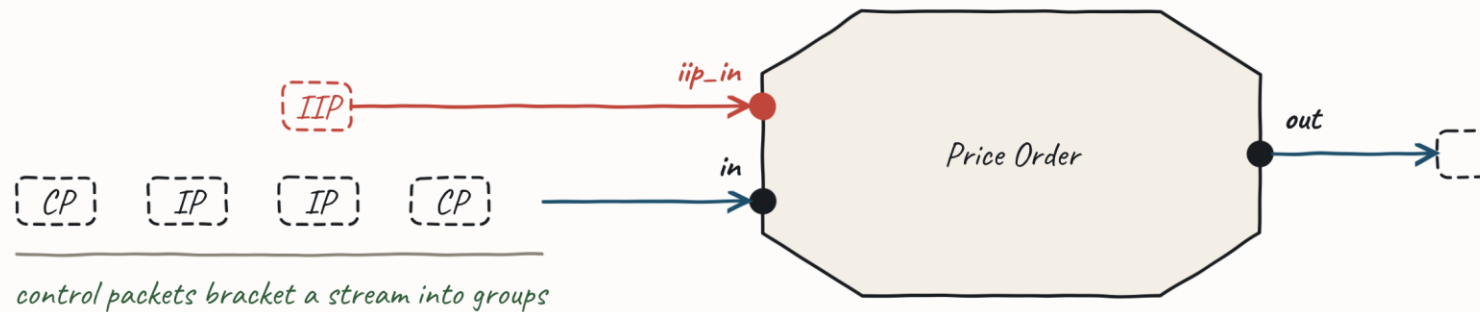


FBP – Initial Information Packets

An **IIP** is a packet a component receives at start-up rather than from an upstream component — configuration, or a starting value. Control packets bracket a stream into groups.

a node that has just started knows nothing — the IIP is the standing instruction pinned above the desk

*configuration, or a starting value —
read once, at start-up*



MQTT retained messages are the same idea: the broker holds the last value on a topic so a late subscriber gets state immediately

FBP — Where Do Lookups Live?

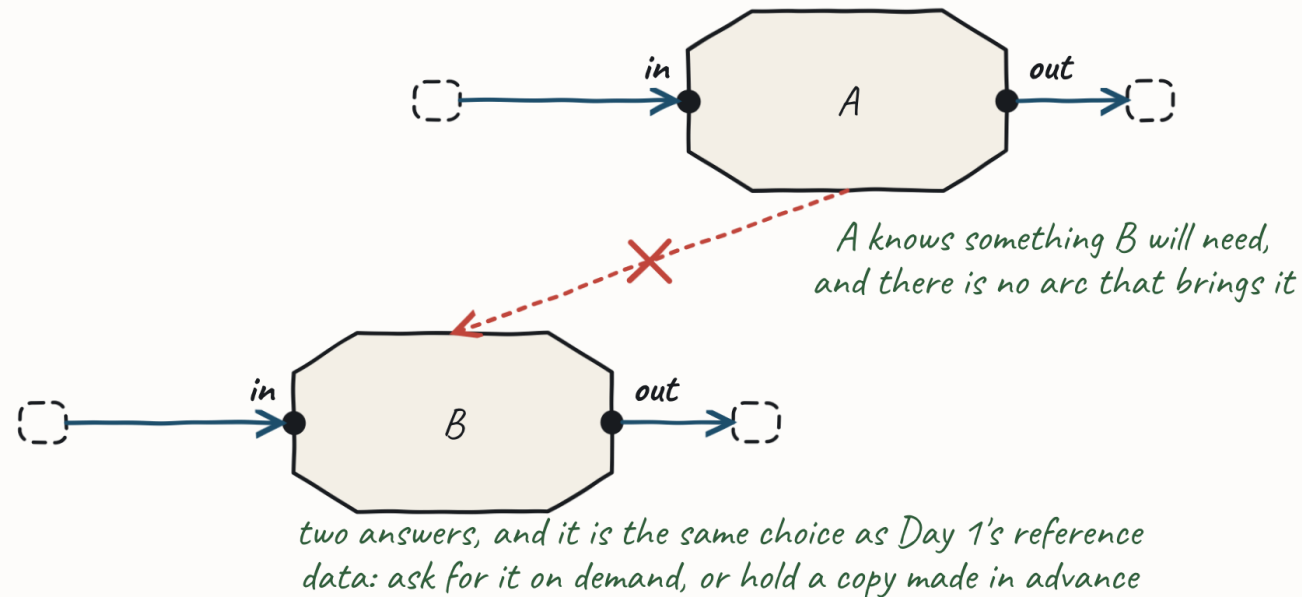
A component needs data it was not sent. Two answers, and it is the same choice as Day 1 §6.2 *Reference Data*.

- **Ask for it.** Add a lookup port: query out, pause, response in. The component stops until the answer comes back — **the walk of shame**. On-demand, and now you are temporally coupled to whoever answers.
- **Build it in advance.** A **Build Lookup** node listens to the stream that owns the data and maintains a table the working component reads locally. In-advance — no pause, no temporal coupling, and the copy is behind by a broker hop.

FBP – Where Do Lookups Live?

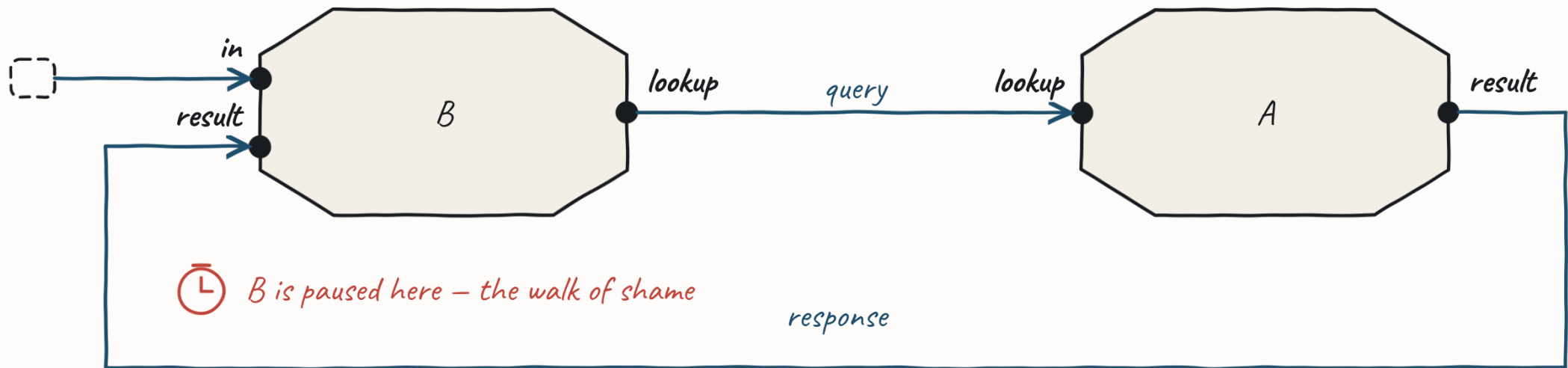
| Same decision as reference data, drawn as a graph — and the same answer: hold the copy.

B has to act on something only A knows — and nothing arriving on B's in-port carries it



FBP – Where Do Lookups Live?

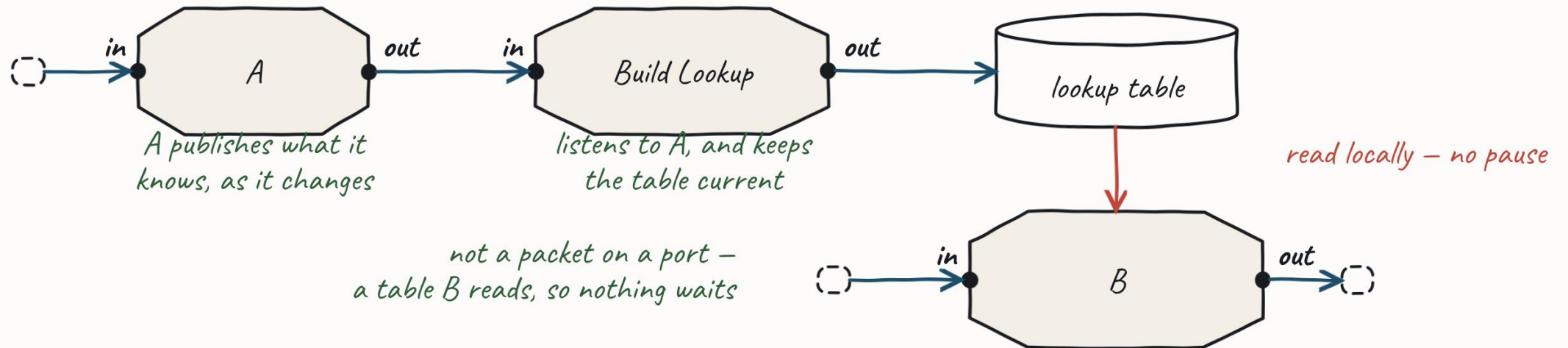
B stops until A answers – on demand, and now B is only as available as A is



use it when the data cannot be replicated. The alternative is to hold a local copy – no wait at all, but the copy is behind by a hop.

FBP – Where Do Lookups Live?

a node whose whole job is keeping a local copy current, so nobody has to walk – that is the Catalogue Maker



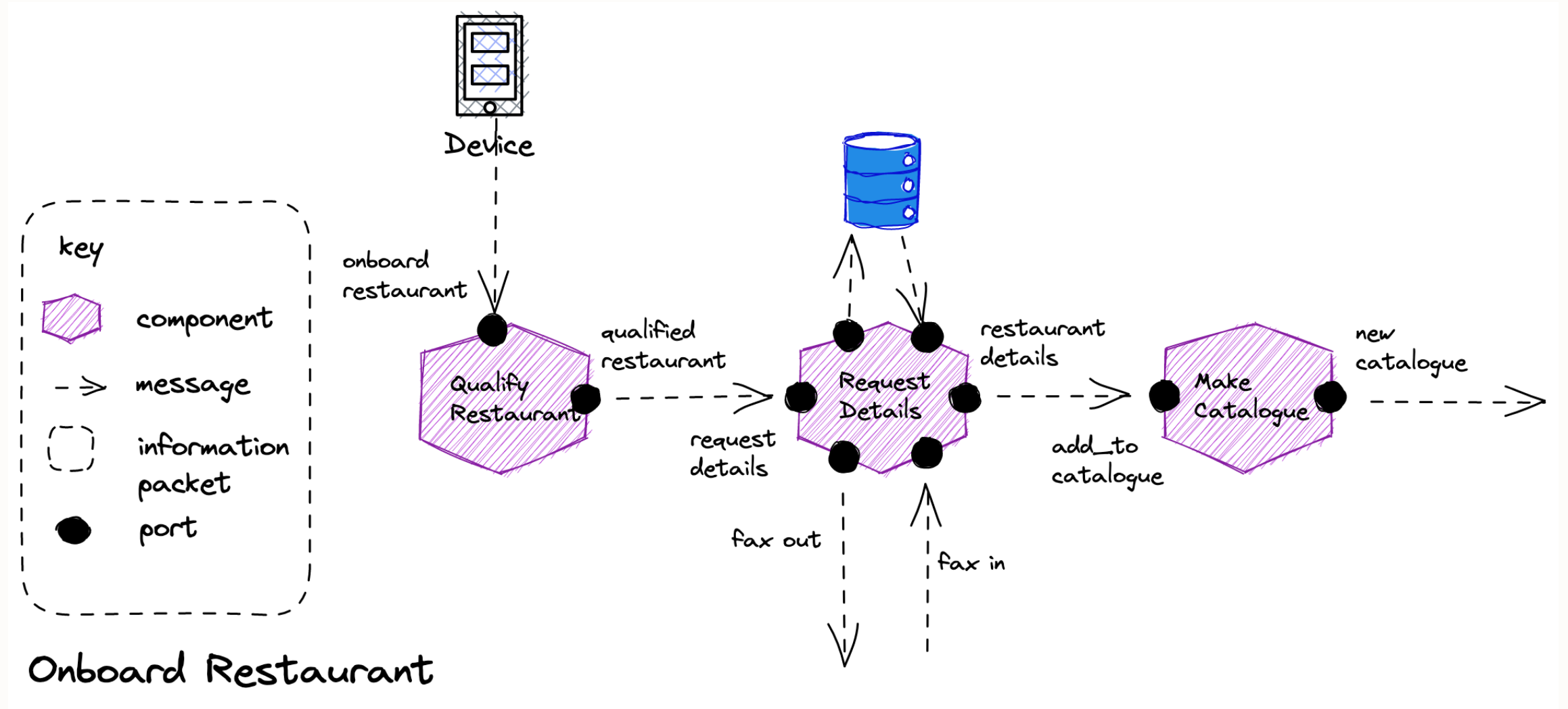
in advance rather than on demand: no temporal coupling, and the copy is behind by one broker hop. The alternative is a lookup port – ask A, and wait.

Worked Example — Onboarding in FBP

The first flow you saw on paper, drawn as a graph. Request/response with an external participant who is a fax machine.

- **Storage.** A component takes work from the `request_details` port but cannot forward to `restaurant_details` without an answer from `fax_in` — so we store the workflow state, in case we crash before the response arrives. *That is filing it before you send it.*
- **Correlation.** To match the response to that saved state we send a **correlation id** on the packet to `fax_out`; the restaurant returns it on the packet from `fax_in`; we use it to look up the stored workflow. *That is the reference number written on the fax.*
- **Lookup.** We build a store from packets raised by another component, to act as the lookup table for information needed to process a request. *That is the Catalogue Maker.*

Worked Example – Onboarding in FBP

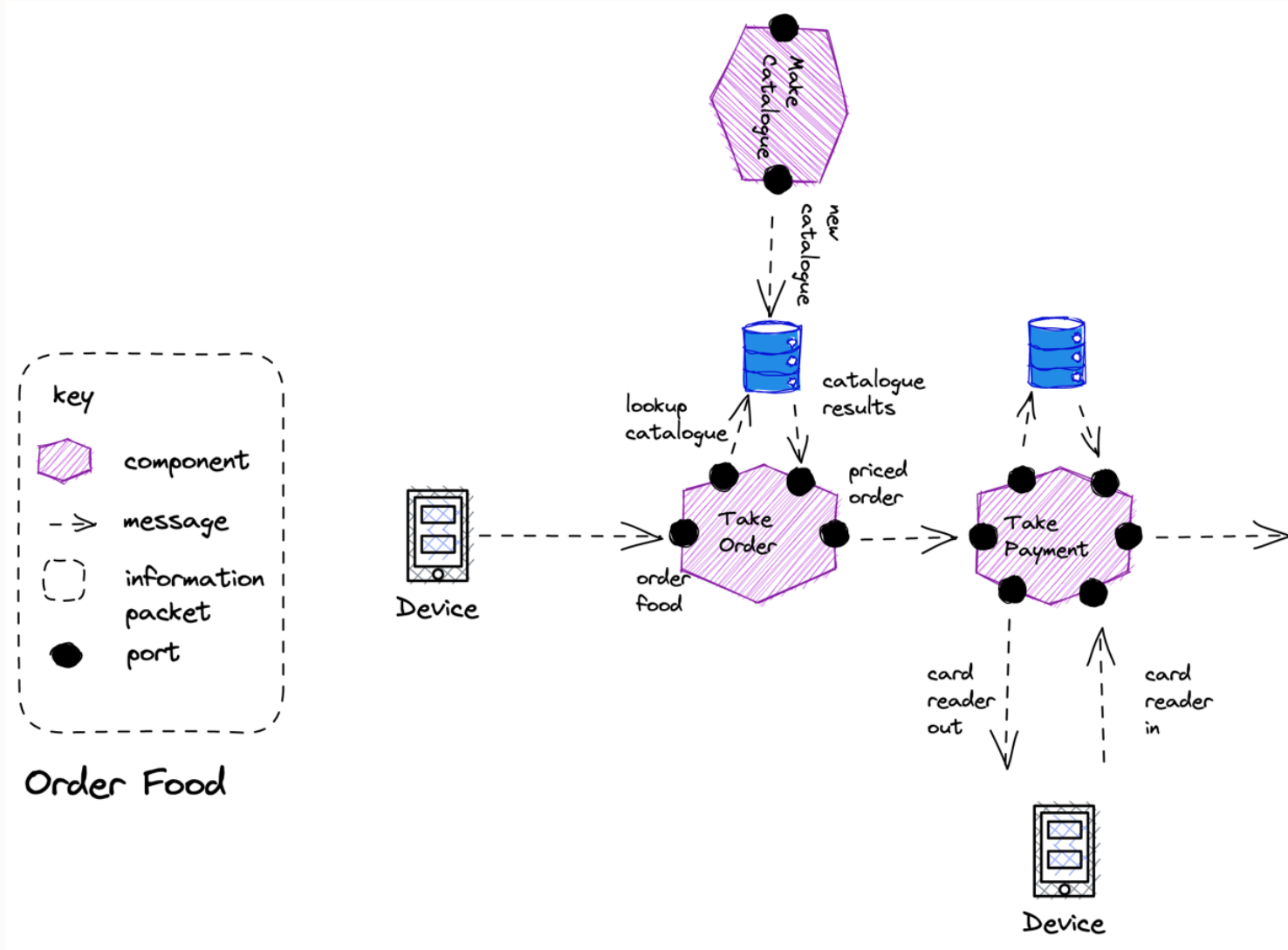


Worked Example — the Order Flow in FBP

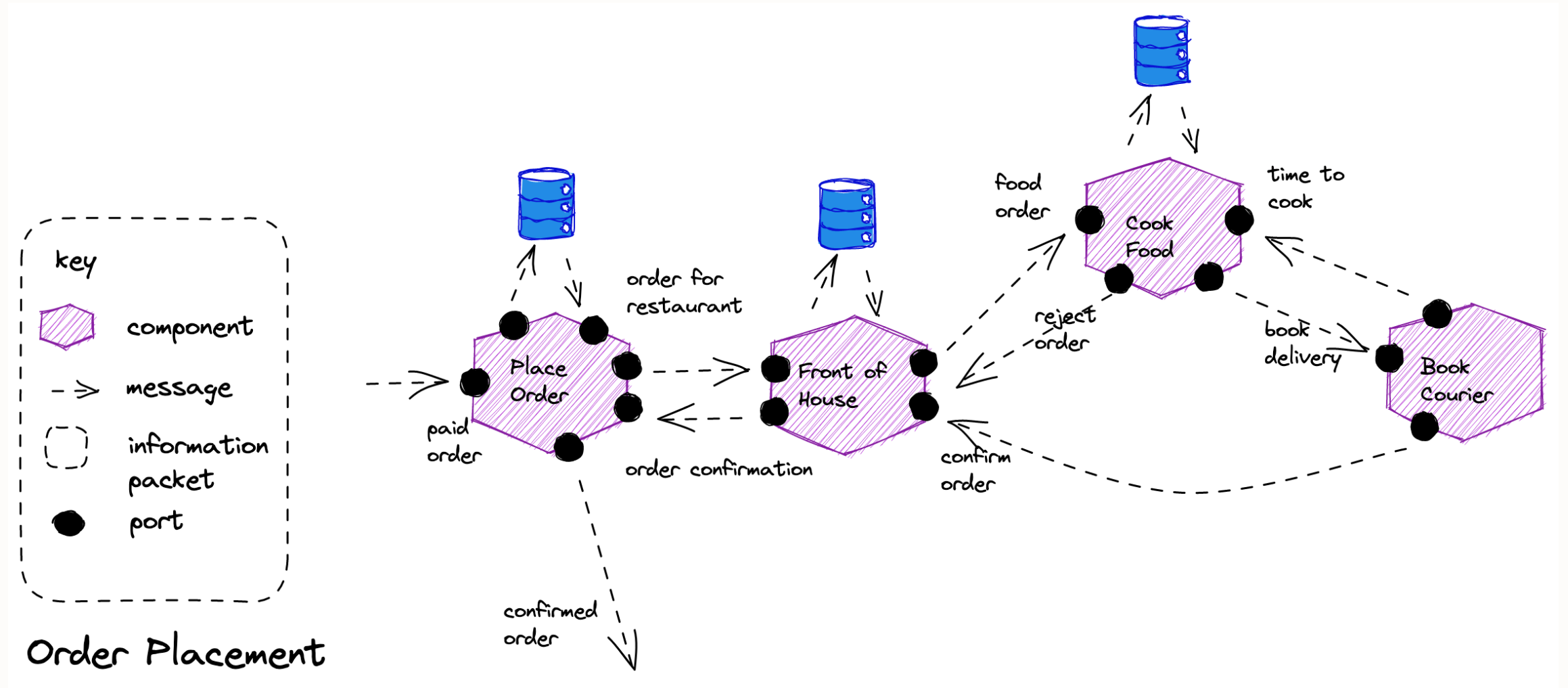
Order, placement and confirmation as one graph. The three flows delegates saw separately on paper compose into a single network — which is the thing paper could not show you.

- **Order Food** — the order enters the network. Ports in, packets out; nobody is called.
- **Order Placement** — the fax hand-off across the organisational boundary, the same store-and-correlate shape as onboarding.
- **The whole network.** Put them together and the confirmation is just another packet arriving on another port. There is no step at which anything is "in charge".

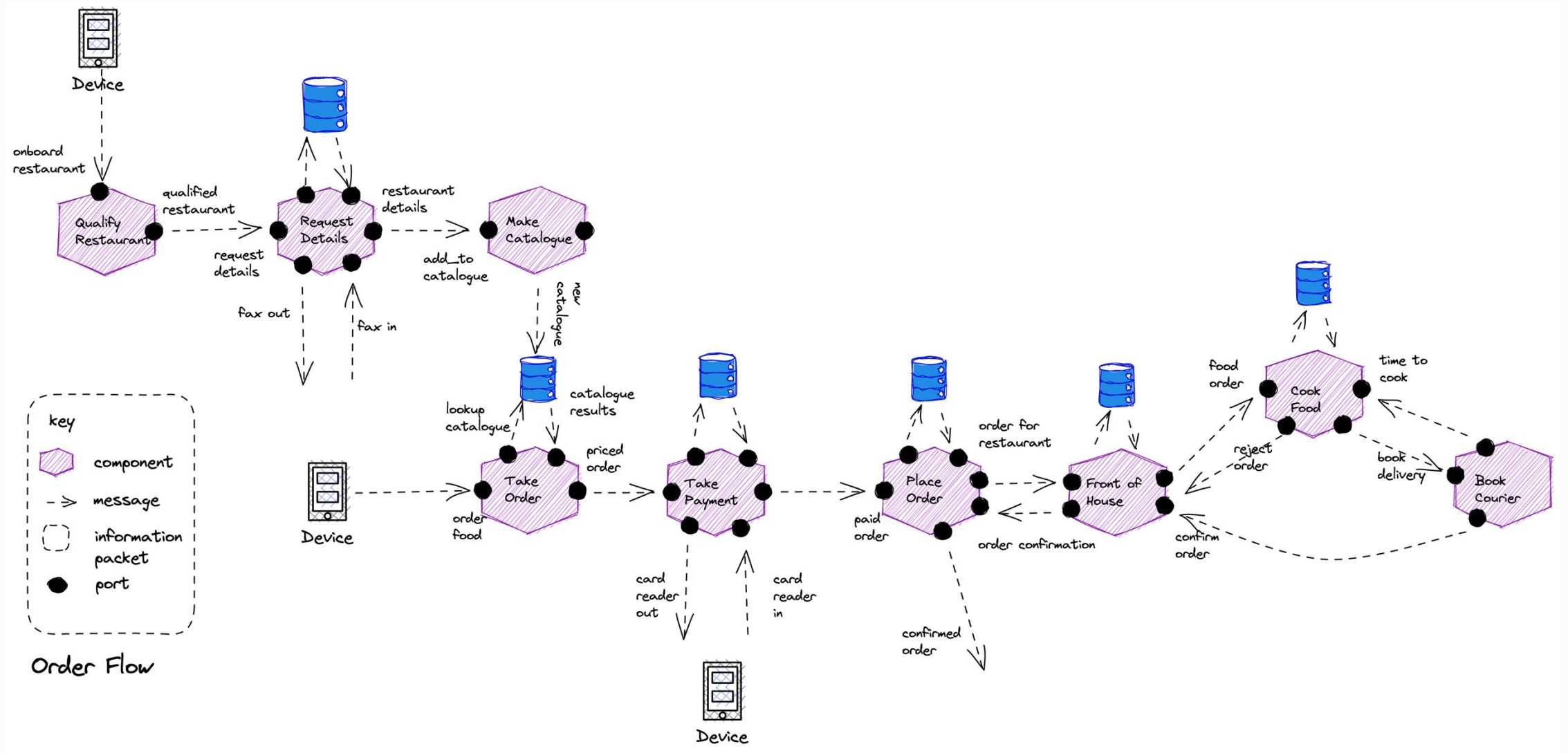
Worked Example – the Order Flow in FBP



Worked Example – the Order Flow in FBP



Worked Example – the Order Flow in FBP



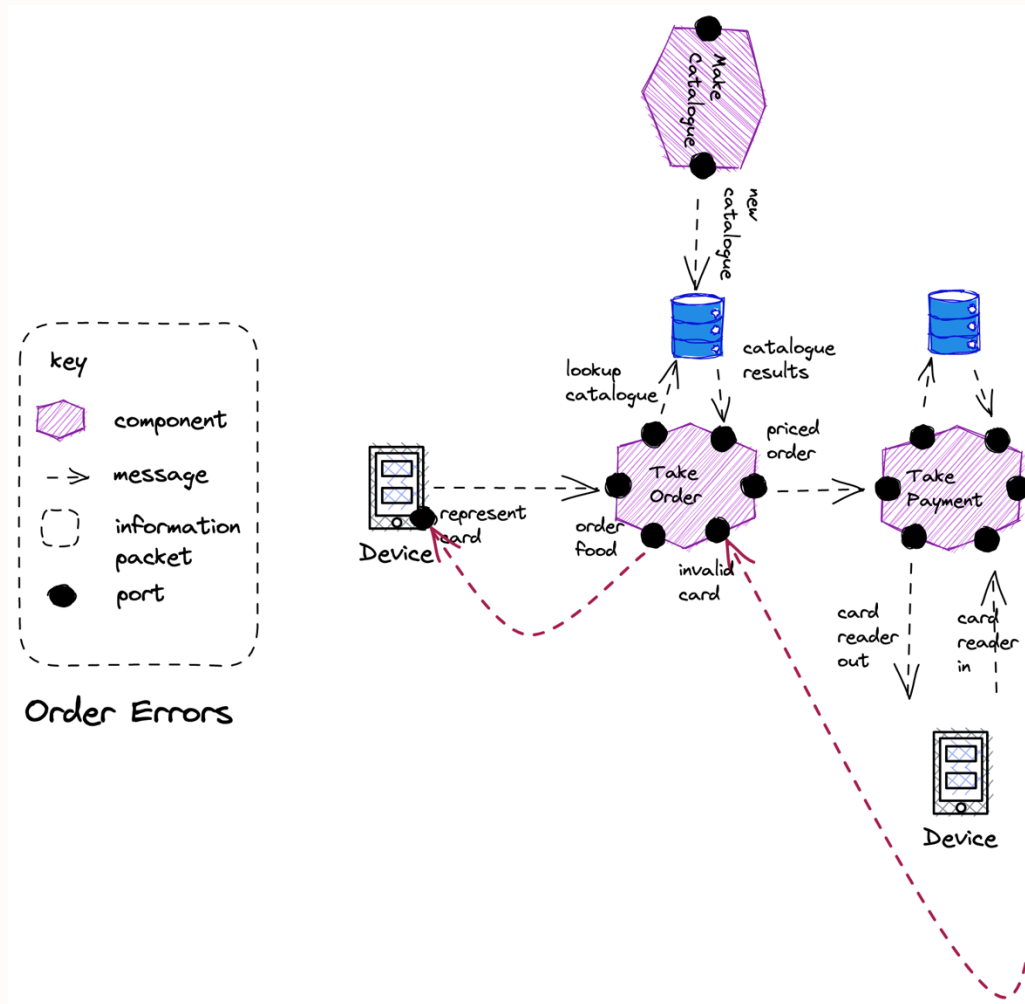
Worked Example — When It Fails, and What the Arcs Really Are

The error variant, as a graph. The same failures from *How Do We Deal with Errors?* — the request goes missing, no receipt comes back, the same document arrives twice — expressed in ports and packets.

- Retry is a packet sent again on the same port; the correlation id is what stops it being a second order.
- The stored workflow is what makes retry safe, because the answer can arrive after a crash.
- The failure is **local to a node**. The graph does not unwind; the packet waits.

Then the last move of the movement: **make the arcs middleware and the nodes processes.**

Worked Example – When It Fails, and What the Arcs Really Are



Worked Example – When It Fails, and What the Arcs Really Are

an arc that survives a crash is a queue; a node that survives a crash is a service



message-oriented middleware

*the connector was always external – the component only ever knew a port name,
so making it a broker changes nothing inside the component*

*everything the room drew on paper this morning, and as a
graph an hour ago, is the thing Day 1 spent a day building*

Now Do One — the Hotel, as a Graph

You have seen the takeaway flow four ways: **as paper**, **as errors on paper**, **as a graph**, and **as a graph that fails**.

Now draw the hotel's as a graph.

- Your own flow from this morning, re-expressed: information packets, nodes, ports.
- Where do the lookups live? Where is state stored? Which arcs must survive a crash?
- Then join your graph to the next table's.



DON'T PANIC

The Name

Reactive: the name for what you have been building since yesterday morning.

The Reactive Manifesto

Reactive derives from **reactive programming**, not from OO.

Published 2014. It defines an architectural style — **Reactive Applications**. Write applications that:

- **react to events** — the event-driven nature enables everything else;
- **react to load** — scalability rather than single-user performance;
- **react to failure** — resilient systems that recover at all levels;
- **react to users** — combine the above into an interactive experience.

Four traits: **Responsive** (responds in a timely manner), **Resilient** (stays responsive in the presence of failure), **Elastic** (stays responsive under varying workload), **Message Driven** (relies on asynchronous message passing). — reactivemanifesto.org

The Reactive Manifesto

You have met two of these already, under other names.

- **Resilient** is what we called **robust** on Day 1: stay working when something you depend on is not.
- And *easy to change* is in the manifesto's own words: "Systems built as Reactive Systems are more flexible, loosely-coupled and scalable. This makes them easier to develop and **amenable to change**."
- **Elastic** and **Responsive** are the two Day 1 did not have names for.
- **Message Driven** is the mechanism — the same one, all the way through.

Reactive Traits — Value, Form, Means

Read the four traits as value, form and means:

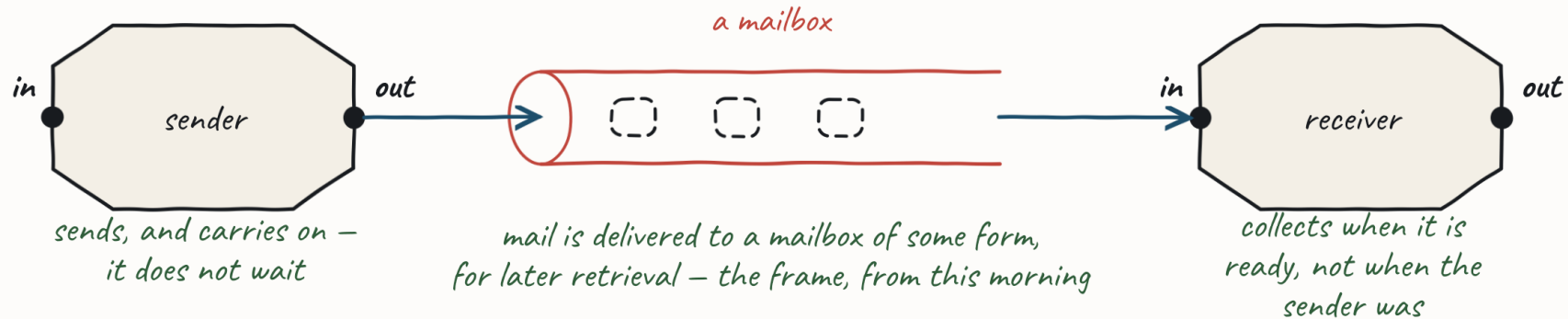
- **Responsive** is the **value** — the thing the user actually gets.
- **Resilient** and **Elastic** are the **form** — the shape that delivers it under failure and load.
- **Message Driven** is the **means** — how the form is achieved.

This split is **Bonér's**, from his talks — the manifesto itself does not rank the four traits this way.

Message Passing

Message passing is an asynchronous method of communication: both parties need not be simultaneously present. Mail is delivered to a **mailbox** of some form, for later retrieval.

both parties need not be present at the same time – because something in the middle holds the message



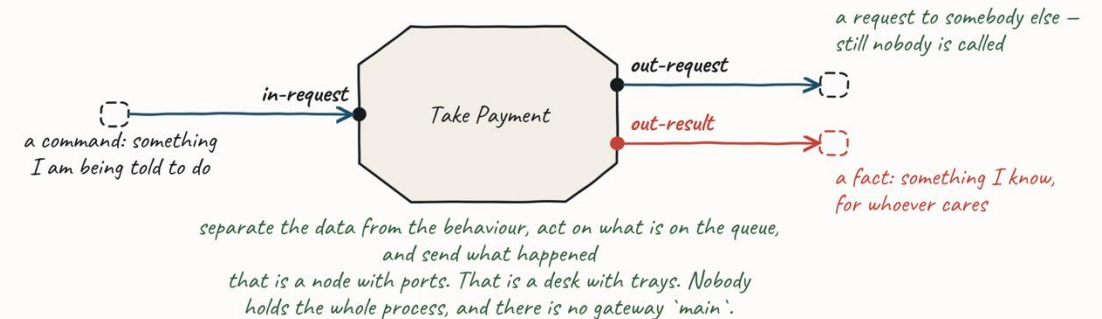
an asynchronous method of communication: the invoker sends, and relies on the receiver to select and run the code that answers it

Microservices Are Reactive Architectures

We separate data from behaviour, activate a component in response to work being on a queue, process it, and send an outgoing message. That is a node with ports. That is a desk with trays.

- A component's inputs are commands and events; its outputs are events.
- Nobody holds the whole process. There is no gateway main.

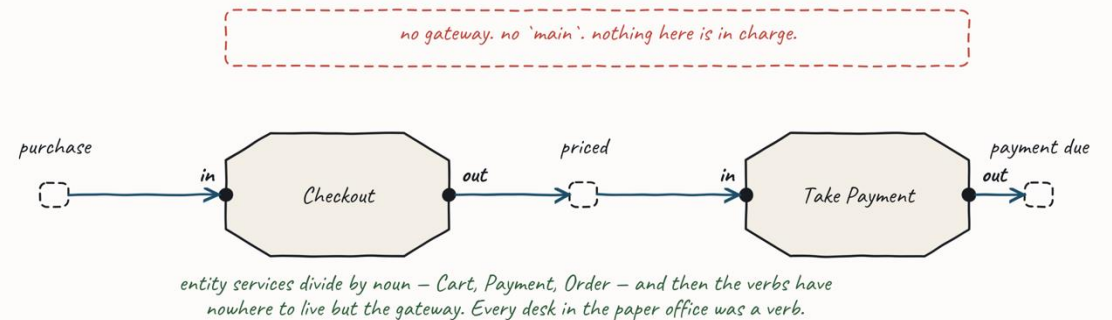
a component's inputs are commands and events; its outputs are events – and nobody is waiting on them



Partitioning and Dataflow

- **Partition to exploit parallelism** — find the tasks that could run concurrently and give each its own component.
- **Coordinate dataflow** — "orchestrate a continuous steady flow of information", dividing the system by **behaviour**, not by structure.

divide by verb, not by noun — Checkout and Take Payment are things the business does, not things it stores



Putting Reactive Together

Four mechanisms are coming, and each one answers a different way of being overwhelmed.

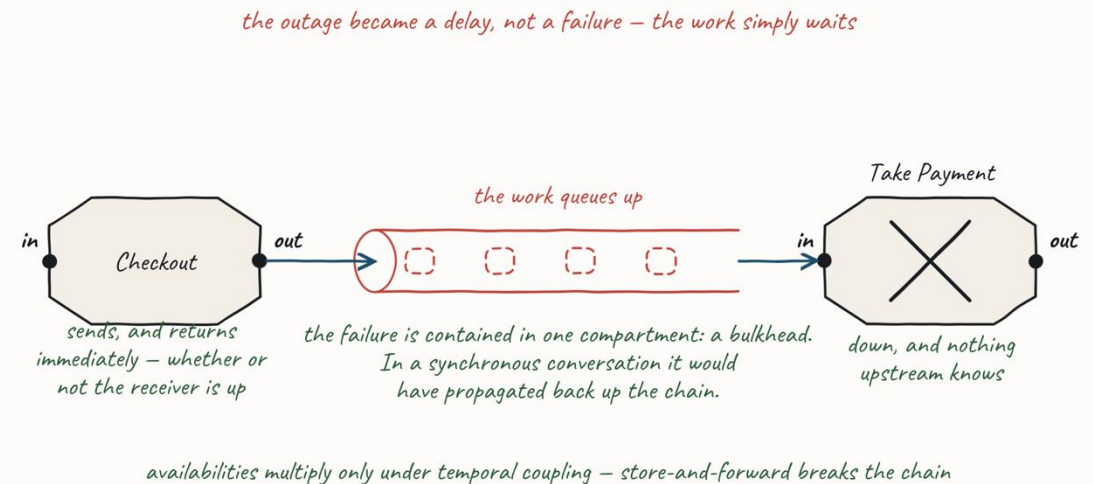
- **Message passing** decouples in time — the bulkhead.
- **Backpressure and load-shedding** handle a consumer that cannot keep up.
- **Circuit breaker** handles a downstream that is failing rather than slow: stop consuming, stop hammering it, resume when it recovers.
- **Scale out, not up** — a supervisor fans work out to worker instances; that is elasticity, and it is competing consumers from Day 1 §4.3.

| Responsive under failure, responsive under load. The traits are these mechanisms.

Bulkheads

In a synchronous conversation both parties must be up, so a fault **propagates** back up the chain and we are brittle. In an asynchronous conversation it does not: the work queues up instead.

- The sender sends and returns immediately, whether or not the receiver is up.
- Messages wait until the receiver asks for them; results go to a queue for collection.
- The failure is contained in one compartment — a **bulkhead**.



When the Pipe Fills — Backpressure or Load-Shedding

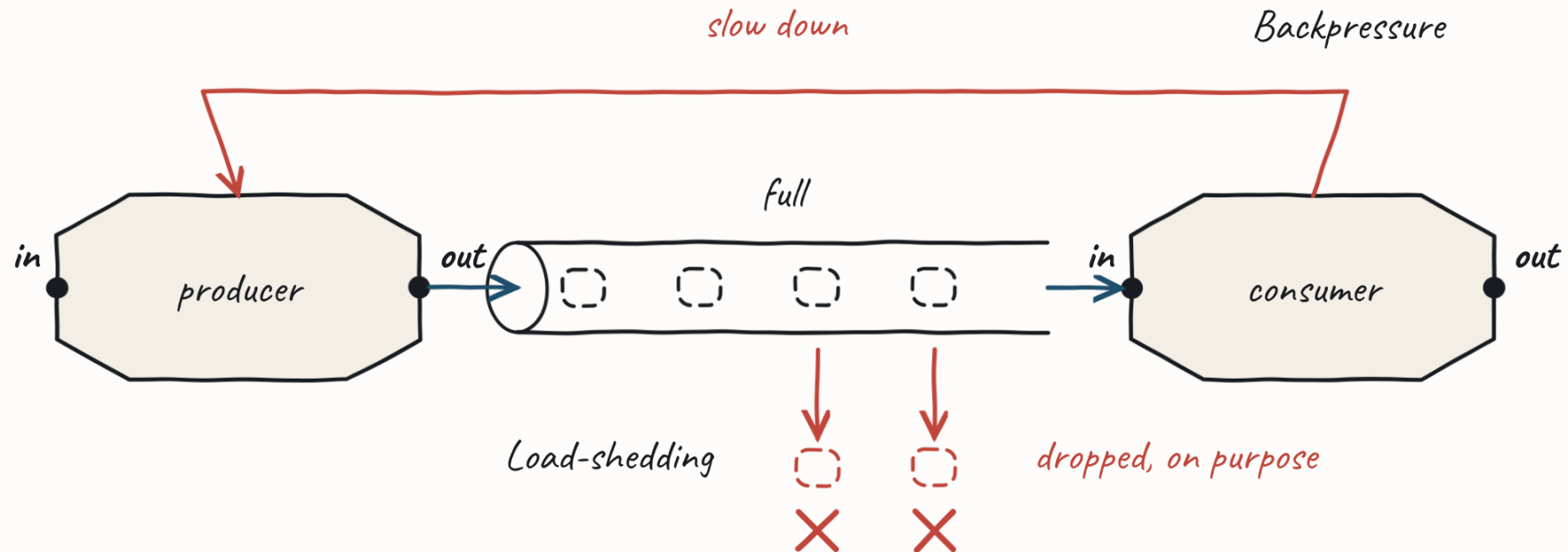
Buffers are finite, so when a consumer cannot keep up something has to give. There are exactly two answers, and it is a **decision**, not a default.

	Backpressure	Load-shedding
what happens	slow the producer down	discard messages
what you lose	latency	data
choose when	data loss is unacceptable and extra latency is tolerable	volume is high and you only need a sample
example	blocking retry against a database you cannot reach	1,000 metrics a second when 10 meets the SLA

- **Backpressure** is the producer feeling backward pressure from the pipe, forcing it to slow until the pressure alleviates. Blocking retry creates it as a side effect: retrying a connection slows consumption, which fills the queue, which slows the producer.
- **Load-shedding** can discriminate — prioritise, and discard only the less valuable data.

When the Pipe Fills – Backpressure or Load-Shedding

the pipe is full, so something has to give – you give latency, or you give data



a system that does neither falls over under load – which is the third answer, and the only one nobody chooses on purpose

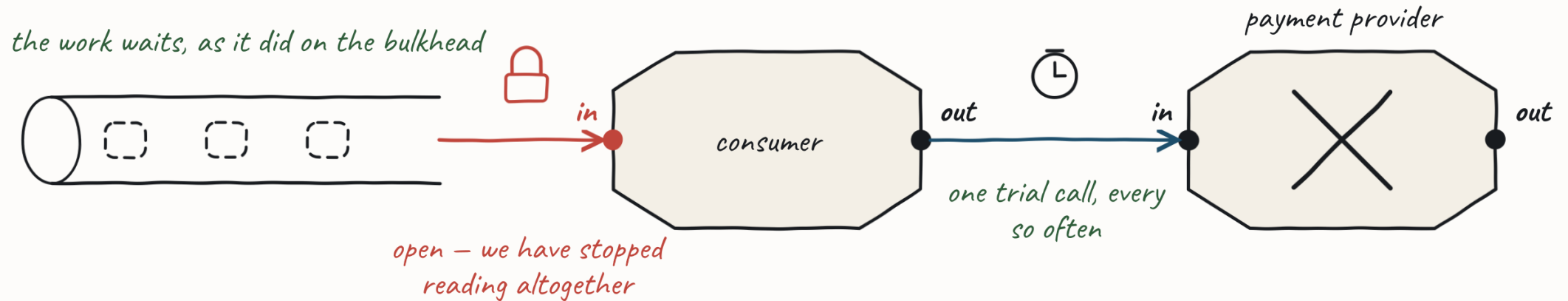
Circuit Breaker

Backpressure answers a consumer that is **slow**. This answers a downstream that is **down** — and that every retry in the system is now making it harder to come back.

- **Closed** — normal. Calls go through, and failures are counted.
- **Open** — the count crossed a threshold, so we stop consuming altogether.
- **Half-open** — after a wait, one trial call. It succeeds and we close; it fails and we open again.

Circuit Breaker

stop consuming, so you stop hammering something that is already down – and let one through now and then to see



the bulkhead answers a downstream that is down; the circuit breaker answers one that is down and being retried into the ground

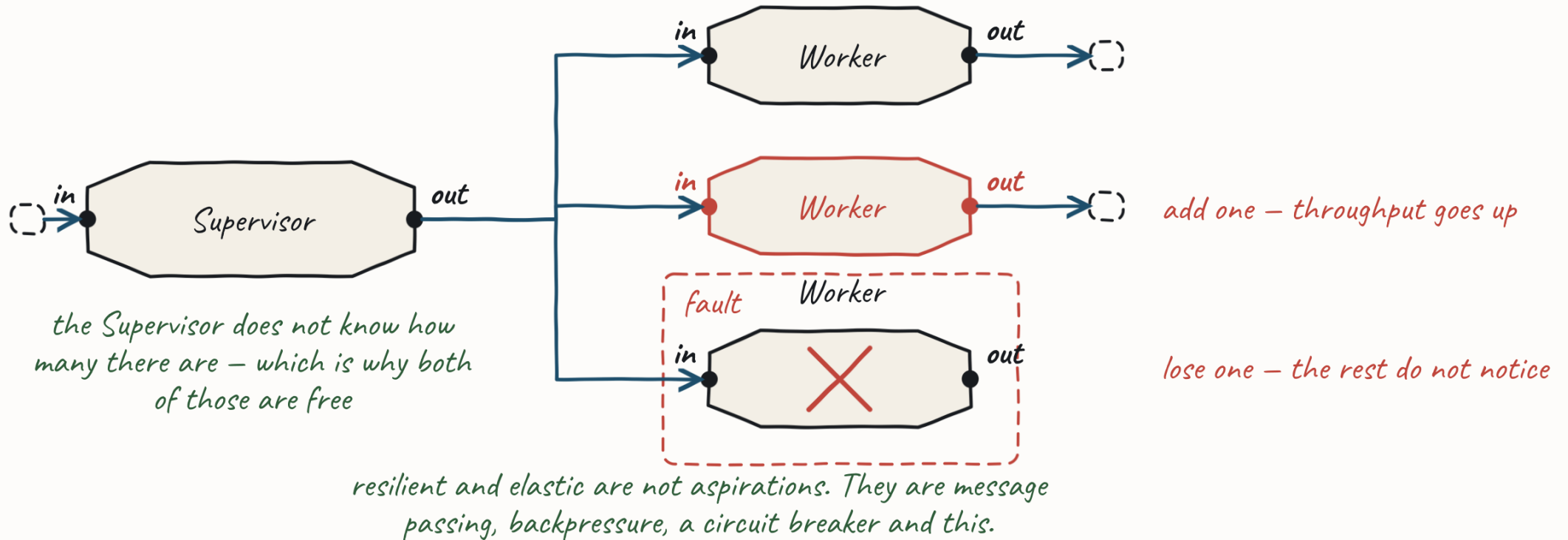
Scale Out, Not Up

The fourth answer is not a mechanism inside one component — it is **more components**. A supervisor fans the same work out to identical workers.

- **Not up.** A bigger machine has a ceiling, and buys nothing at all when the machine dies.
- **Out.** Throughput is a count you can change while the system is running.
- One arrangement, two traits: **Elastic** going up, **Resilient** going down.

Scale Out, Not Up

workers are interchangeable – so you can add one, and so you can lose one



So Who Is in Charge?

The section opened on one question. Here it is again, and now you can answer it — but answer it with people, not slides.

Take your flow from this morning and run it twice.

- **With a conductor.** One person holds a routing slip and tells each desk when to act.
- **With none.** Every desk acts on what is in its in-tray, and nothing else.

Then: **where did knowledge of the whole process live?** And **who had to change when we added a step?**

You have just invented orchestration and choreography. The next section gives them their names — and a notation.

Process Automation

Your Flow, in the Standard Notation

Pre-Arrival, as you modelled it on paper — and the same flow as BPMN. Same three participants, same hand-offs, same numbered steps:

what you drew	what it is called
a desk	a task , sitting in a lane
the heavy vertical bar	a pool boundary
a hand-off between two desks inside the same bar — out-tray to in-tray	a sequence flow
a hand-off across the bar — the phone call, the fax, the mail cart	a message flow
a folder	the state a task reads and writes
the conductor holding the routing slip	the process , and its token

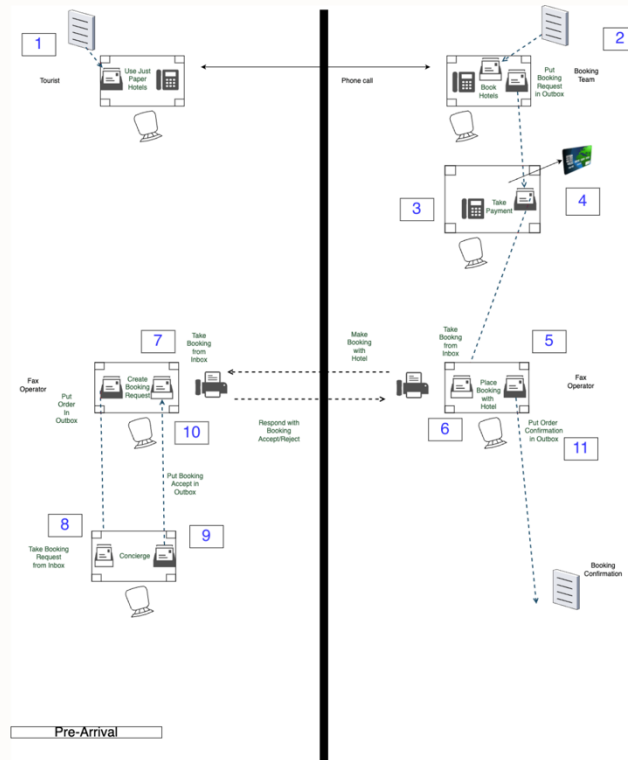
The bar is the discriminator, not the arrow style. Inside it, control walks from desk to desk and the process holds together. Across it, a message leaves and nobody controls what happens next.

Nothing new happens in this section. It gives names to what the room already built.

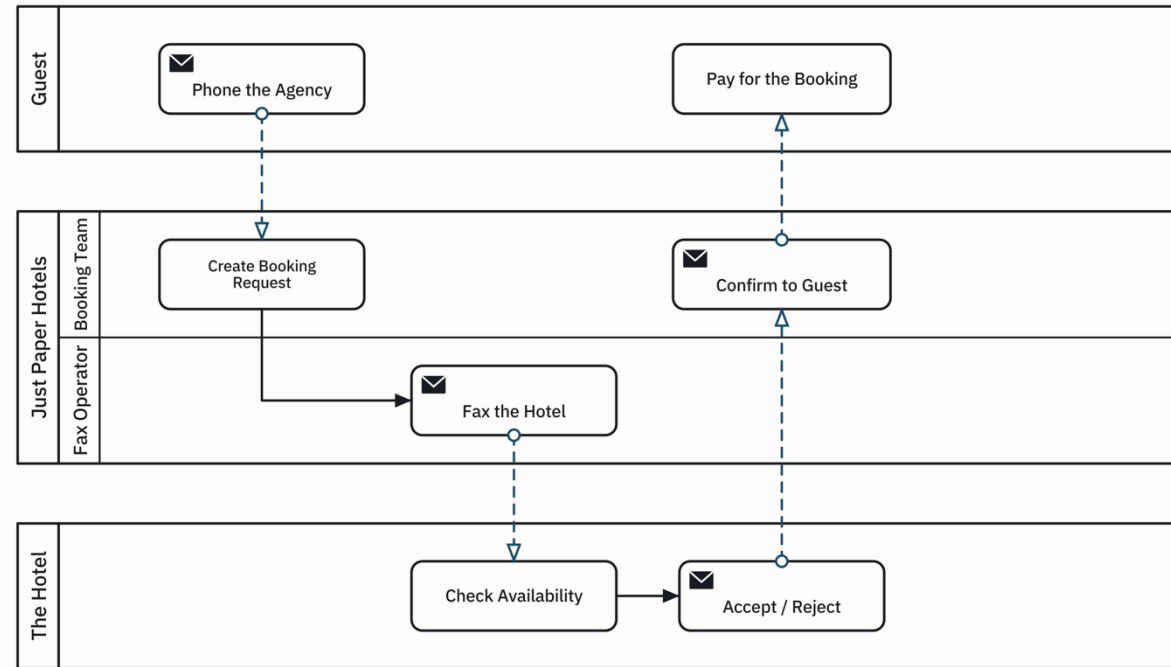
Your Flow, in the Standard Notation

You drew this an hour ago. Here it is again, in a notation the rest of the industry already reads.

you already had every concept – this is only the vocabulary



the desk, the tray, the heavy bar

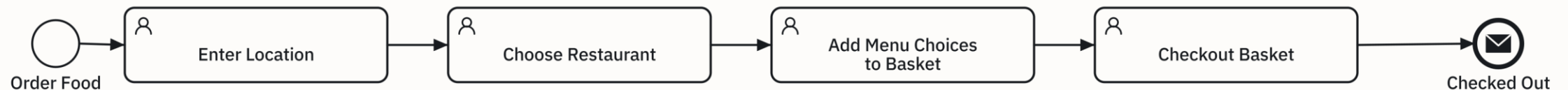


the task, the sequence flow, the pool

BPMN

BPMN (Business Process Management and Notation) is a visual language for diagramming business processes clearly, in a standardized and comprehensive way.

- A **start event** begins a flow; an **end event** terminates it (and may throw a message).
- **Sequence flow** arrows show the path of execution.
- An **event** begins, ends, or interrupts a flow; a **task** is where work gets done.

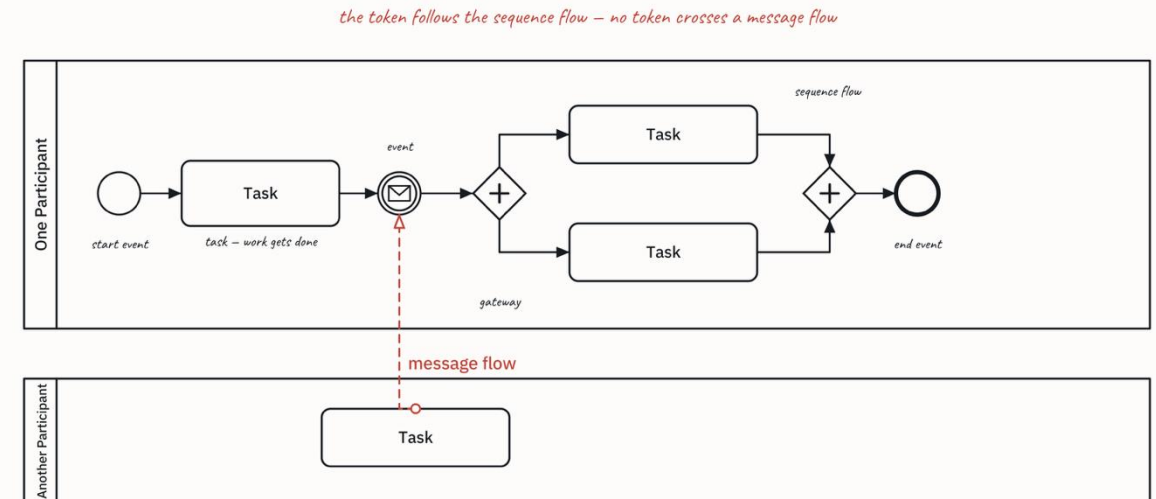


BPMN – The Elements

A BPMN diagram is six things: **Start Event**, **End Event**, **Activity** (Task or Sub-process), **Gateway**, **Event**, and the arrows that join them.

There are two kinds of arrow, and the difference between them is what the rest of this section is about:

- **Sequence Flow** — control moving *within* one participant. **The token follows it.**
- **Message Flow** — a message crossing *between* participants. **No token crosses it.**



BPMN — Tasks, Events and Gateways

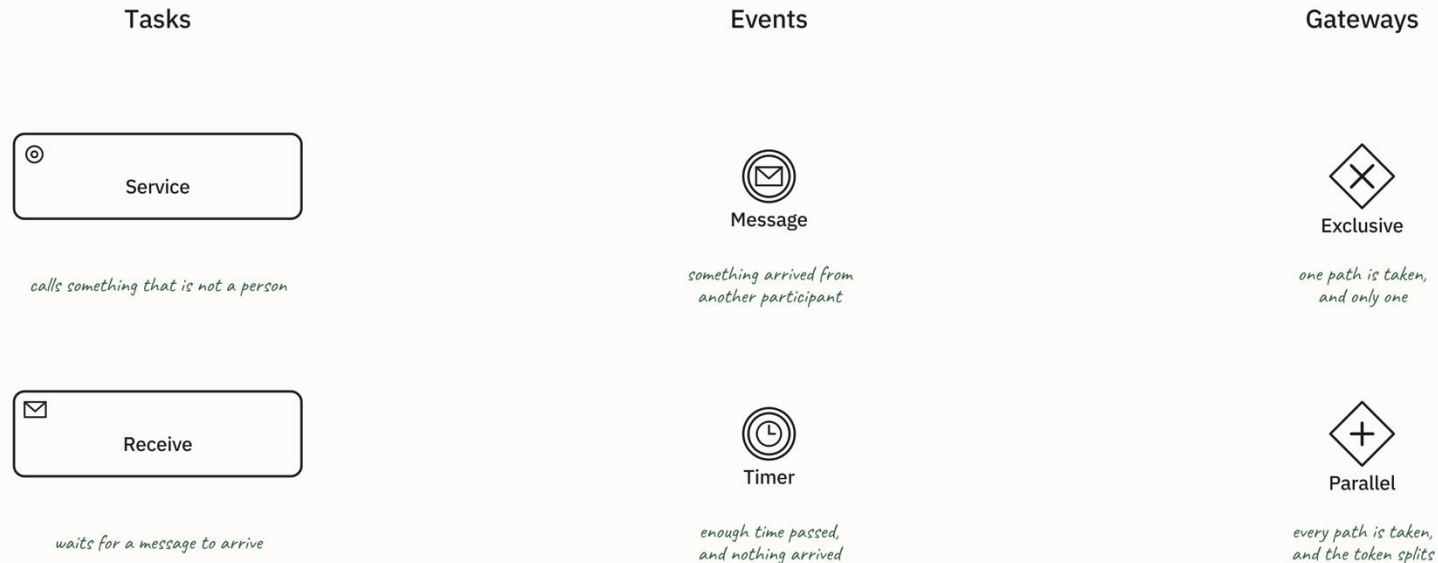
Three of the six have variants. They are reference, not material:

- **Tasks** — atomic activities: Generic, **Service** (uses a service), **Receive** (waits for a message), **Send**, **User** (human, via software), Manual (human, not via software), Business Rule, Script. Markers: Loop, Transaction.
- **Events** — start, end, or interrupt a flow: None, **Message**, **Time**, Signal, **Compensation**, Conditional, Escalation, Parallel, Cancel.
- **Gateways** — branch and converge sequence flow: **Exclusive** (X, one path), Inclusive, **Parallel** (+, split/join), Complex, Event-based (an event picks the path).

BPMN – Tasks, Events and Gateways

Six of them do nearly all the work: **Service** and **Receive** tasks, **Message** and **Timer** events, **Exclusive** and **Parallel** gateways.

six of them do nearly all the work



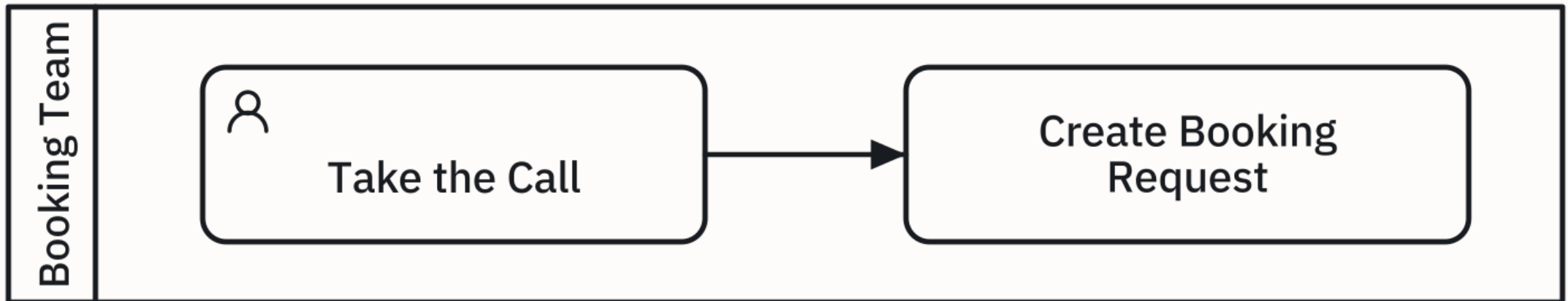
every other task type, event and gateway is on the reference card in your pack

Workflow Patterns

Workflow Patterns — van der Aalst, ter Hofstede, Kiepuszewski, Barros (2000): 5 basic + 15 advanced patterns. *Workflow Control-Flow Patterns: A Revised View* — van der Aalst, Mulyar, Russell, ter Hofstede (2007): 23 new patterns.

Pattern 1 – Sequence

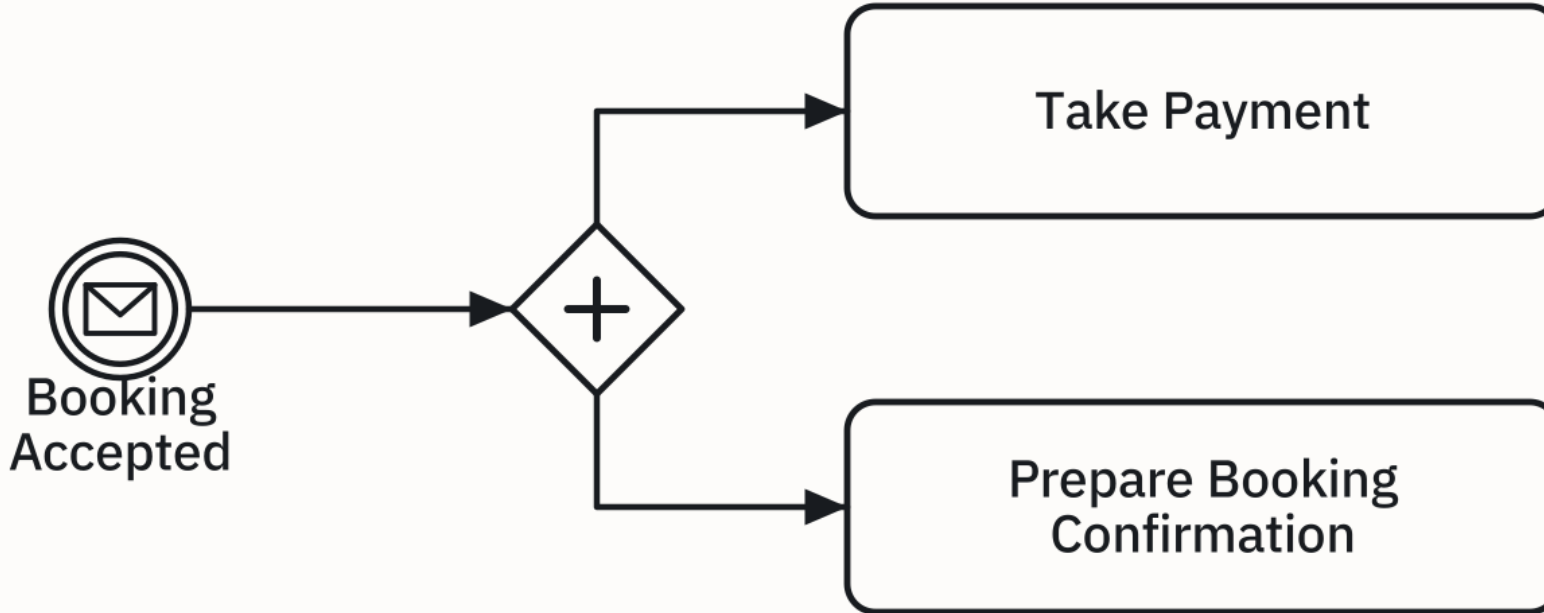
One activity follows another. BPMN: connect two tasks with a sequence-flow arrow. Easy to model and execute. Example (Booking Team): Take the Call → Create Booking Request.



one activity follows another – a sequence flow, and the token moves with it

Pattern 2 – Parallel Split

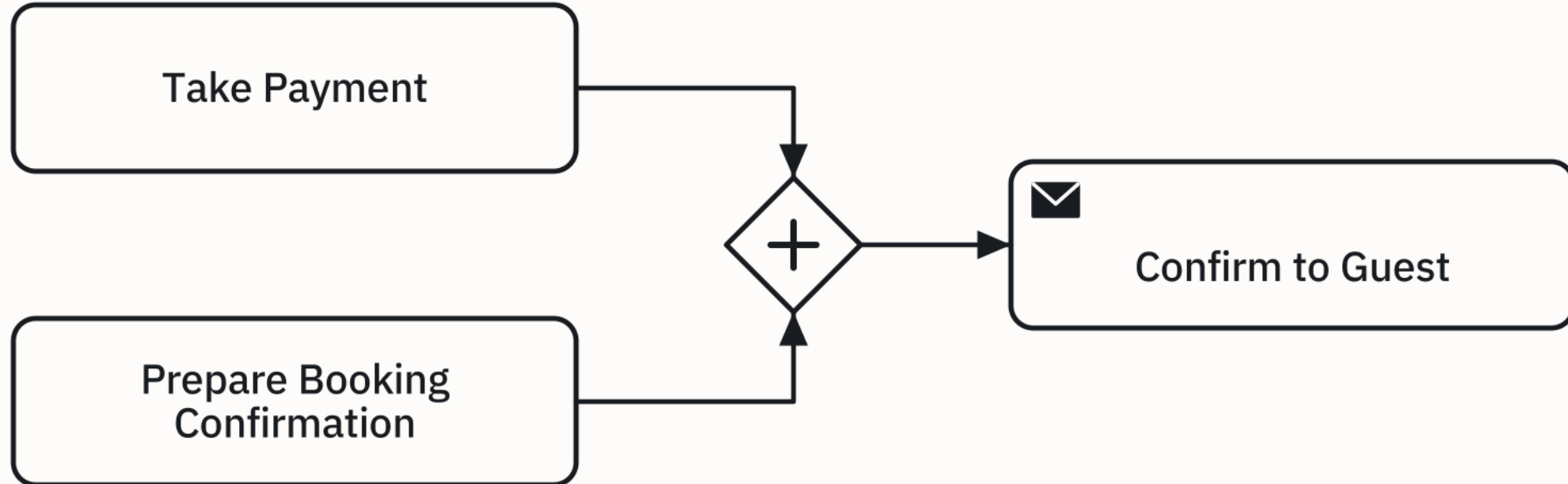
One path splits into two or more concurrent branches. BPMN: a **Parallel Gateway** (+) forks. Example: once the hotel accepts, Take Payment + Prepare Booking Confirmation.



the gateway forks: both branches run, and the token splits in two

Pattern 3 – Synchronization (Join)

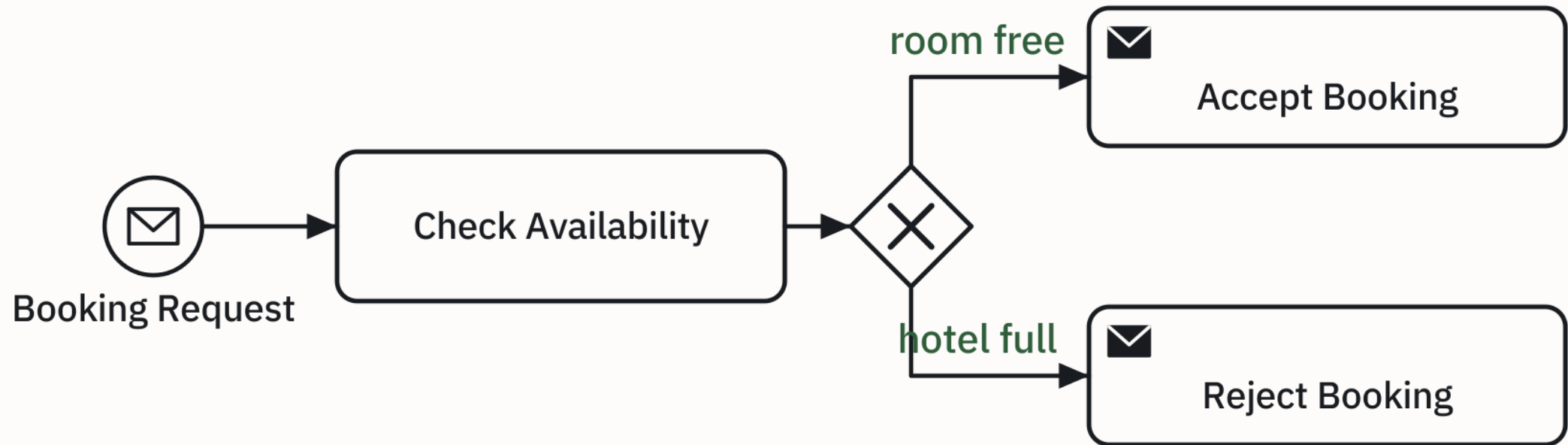
Wait until multiple concurrent branches complete. BPMN: a **Parallel Gateway** joins. Example: the guest is not told until payment has cleared *and* the confirmation is ready.



the join waits for both – the guest is not told until payment has cleared and the confirmation is ready

Pattern 4 – Exclusive Choice

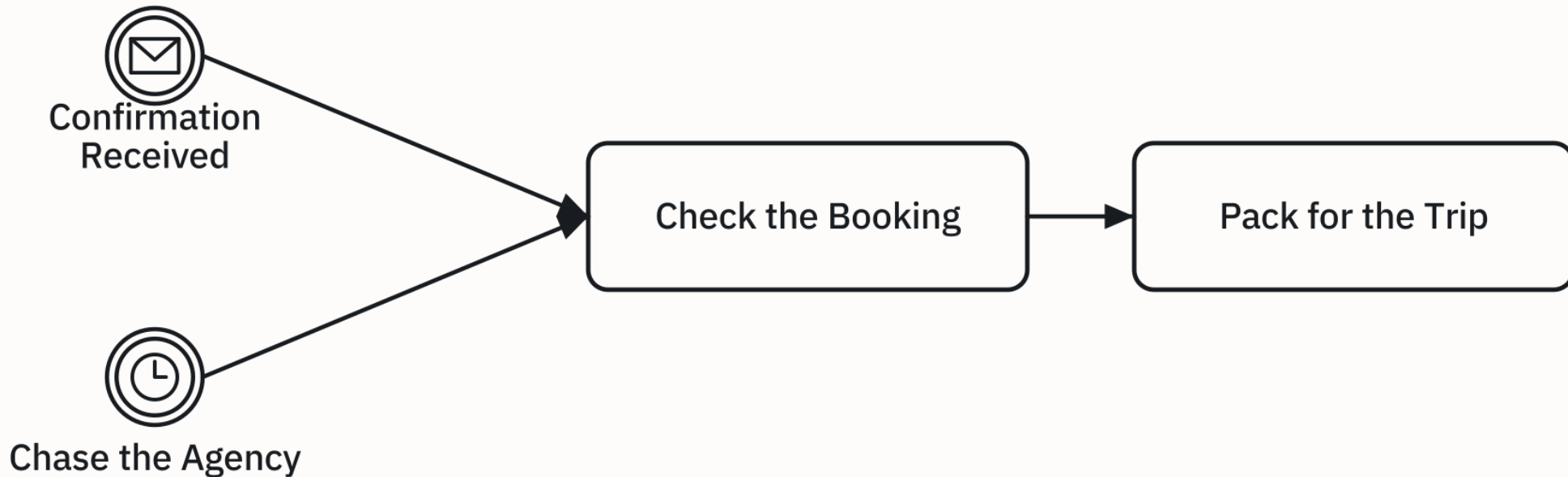
Choose one path based on a condition. BPMN: an **Exclusive Gateway** (X) with condition expressions. Example (Concierge): Check Availability → Accept Booking **X** Reject Booking.



one path only – this is step 7 on the flow you drew, Respond with Booking Accept/Reject

Pattern 5 – Simple Merge

Merge non-concurrent paths back into one. BPMN: a converging sequence flow — no gateway needed if no synchronization is required. Example (Guest): the confirmation arrives, or the guest gets tired of waiting and chases; either way, Check the Booking → Pack for the Trip.



no gateway: the paths were never concurrent, so there is nothing to synchronise

Process = Orchestration

A **Process** describes a sequence/flow of activities. In BPMN it is a graph of flow elements (a sequence flow of activities, events, gateways).

A Process is an **orchestration** — focused on a **single participant's perspective**:

- The sequence flow represents control of a process. Like writing your own script.
- The **token** represents state for an instance.
- Control flow, state and decisions are **all local to the orchestrator**.
- Generally a process orchestration lives within an address space (not distributed) — an embedded workflow or an external process manager, often a state machine or a workflow engine.

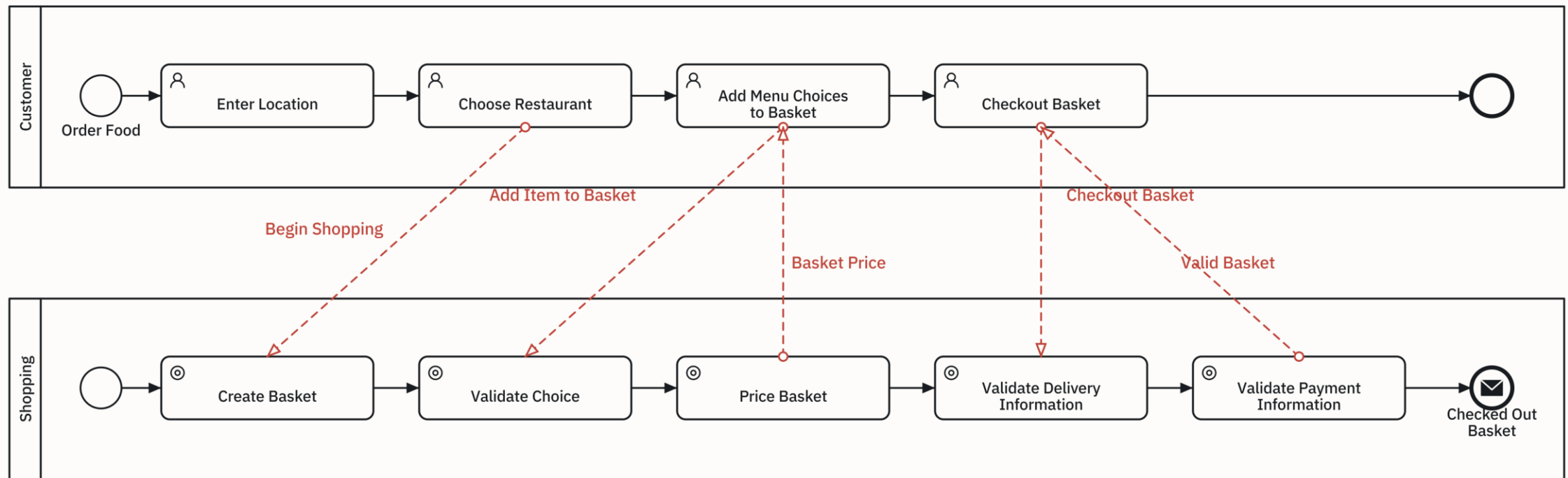
one participant holds the whole process – control, state and the decisions are all local to it



Tokens

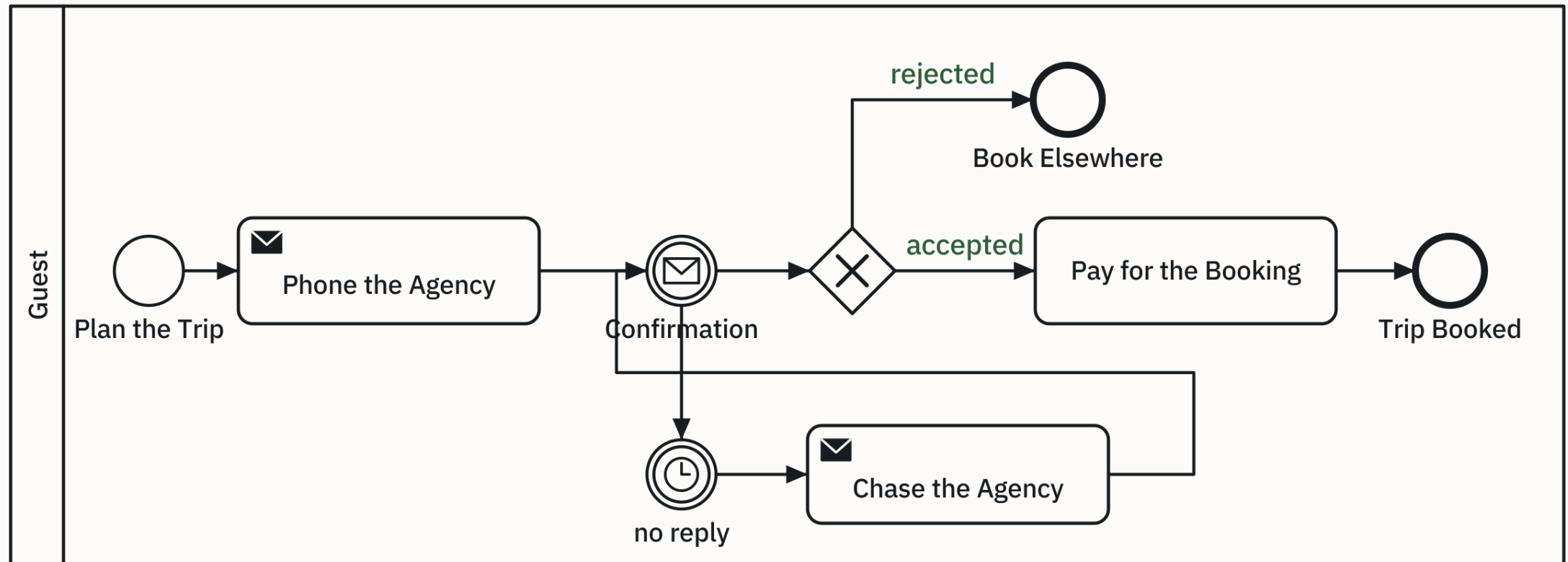
- A **token** represents an instance of a process.
- A token flows down the process; it is the state of the process for that instance.

each pool runs its own sequence flow; the only thing crossing between them is a message



Hotel Example – BPMN Orchestration (Guest Pool)

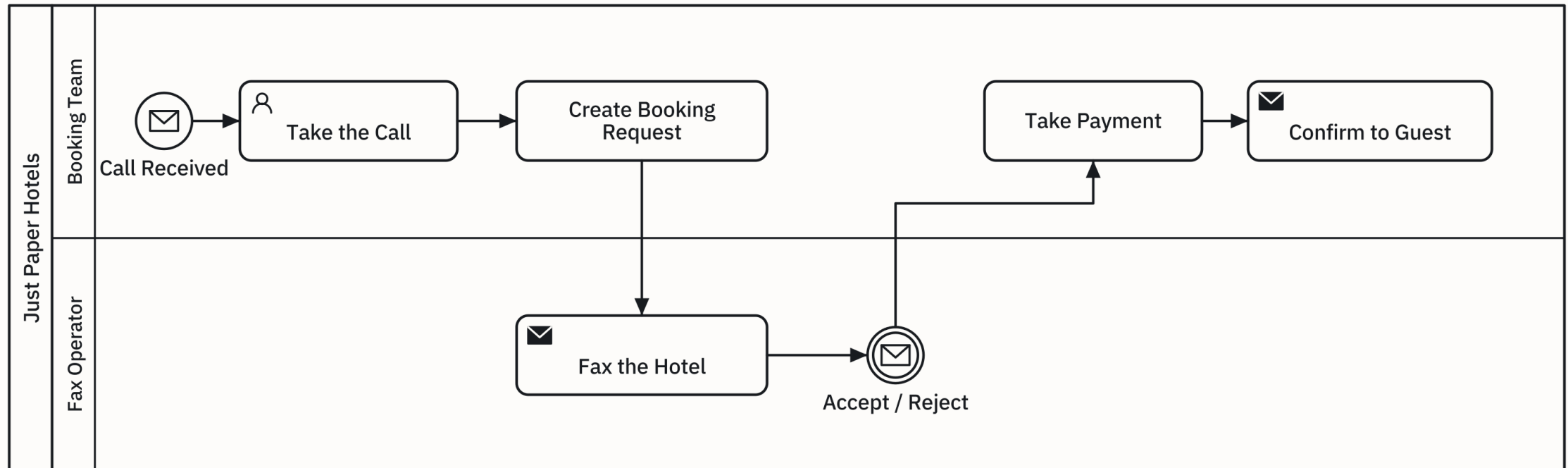
The guest's journey as an orchestration.



one participant, one token, all of the control local to it – this is the conductor from round 4

Hotel Example – BPMN Orchestration (Just Paper Hotels Pool)

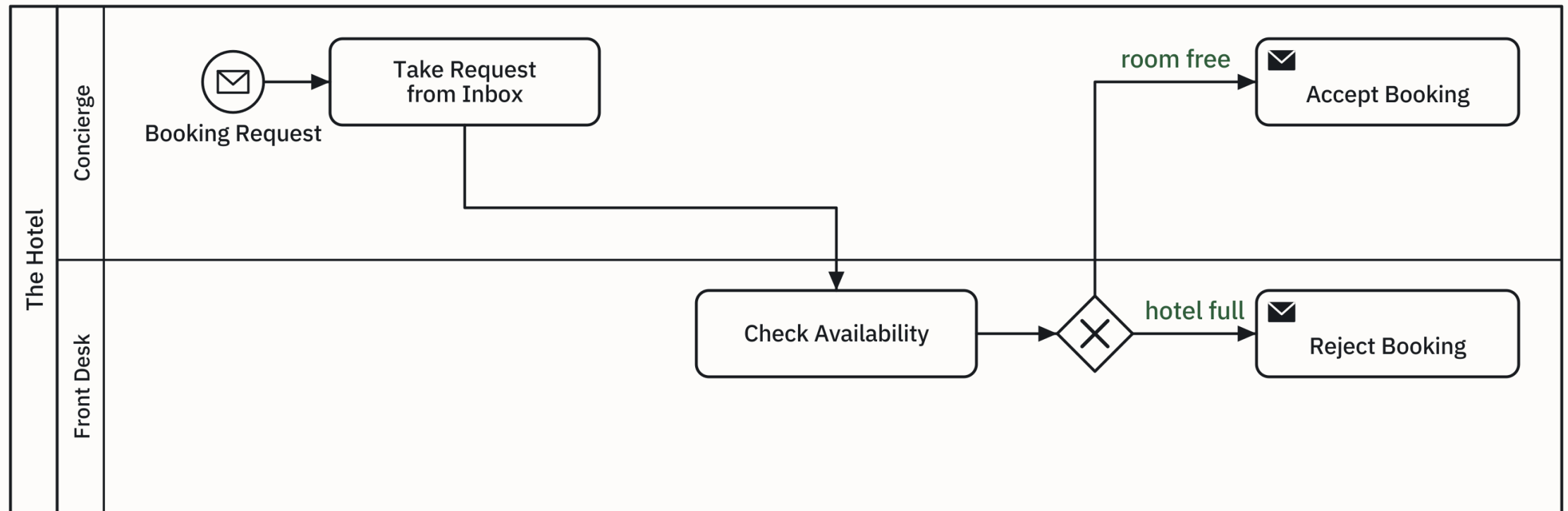
The agency as an orchestration, initiated by a message. Two lanes: **Booking Team** and **Fax Operator**.



two lanes, one token: the fax operator and the booking team are the same participant, so control flows between them

Hotel Example – BPMN Orchestration (The Hotel Pool)

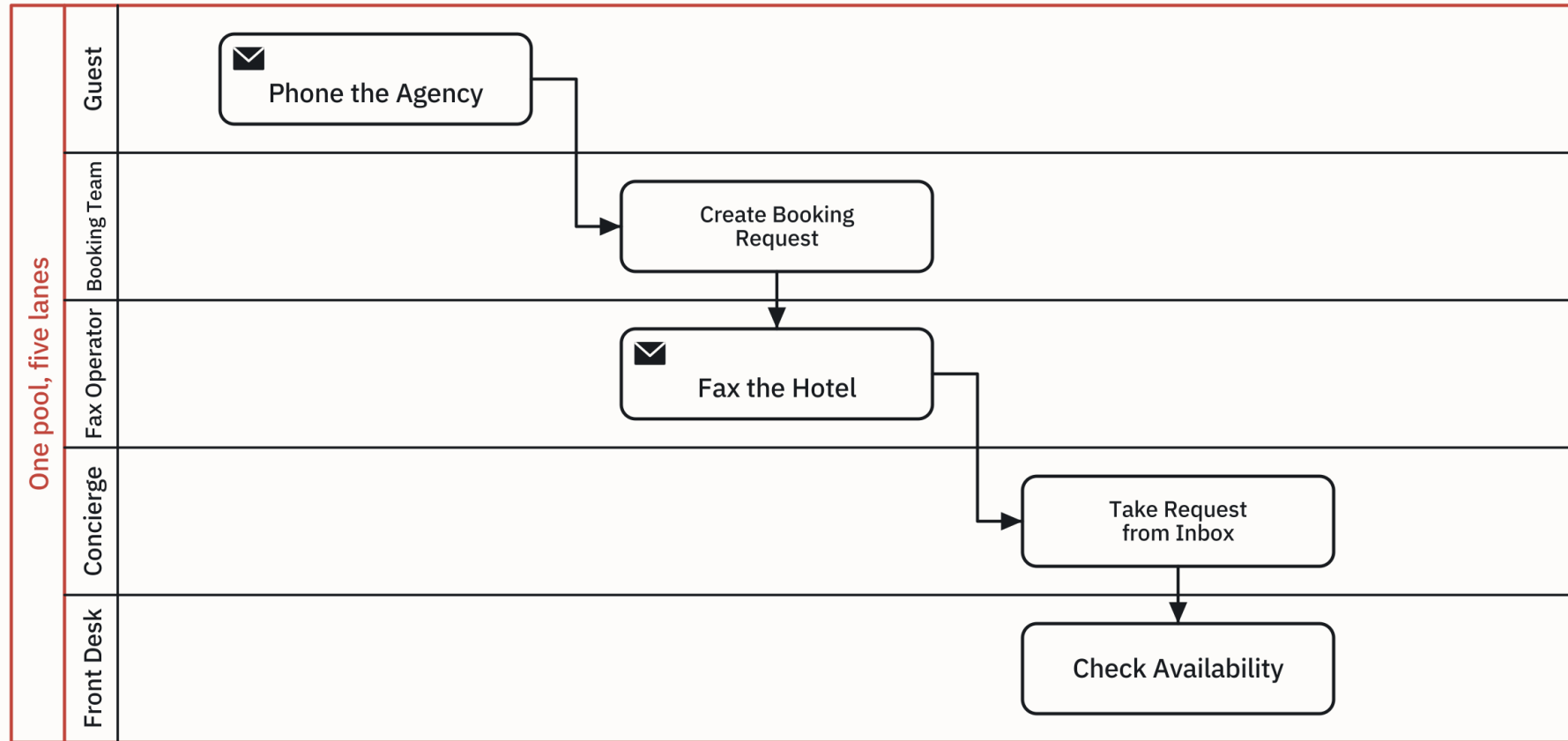
The hotel as an orchestration, initiated by a message. Lanes: **Concierge** and **Front Desk**.



the same shape a third time – which is the point of showing it a third time

Hotel Example – Pools and Lanes

a message event refers to a message from outside – and drawn this way, there is no outside



some tasks here are about the interaction, others are oblivious to it. One token cannot be both.

Collaboration and Choreography

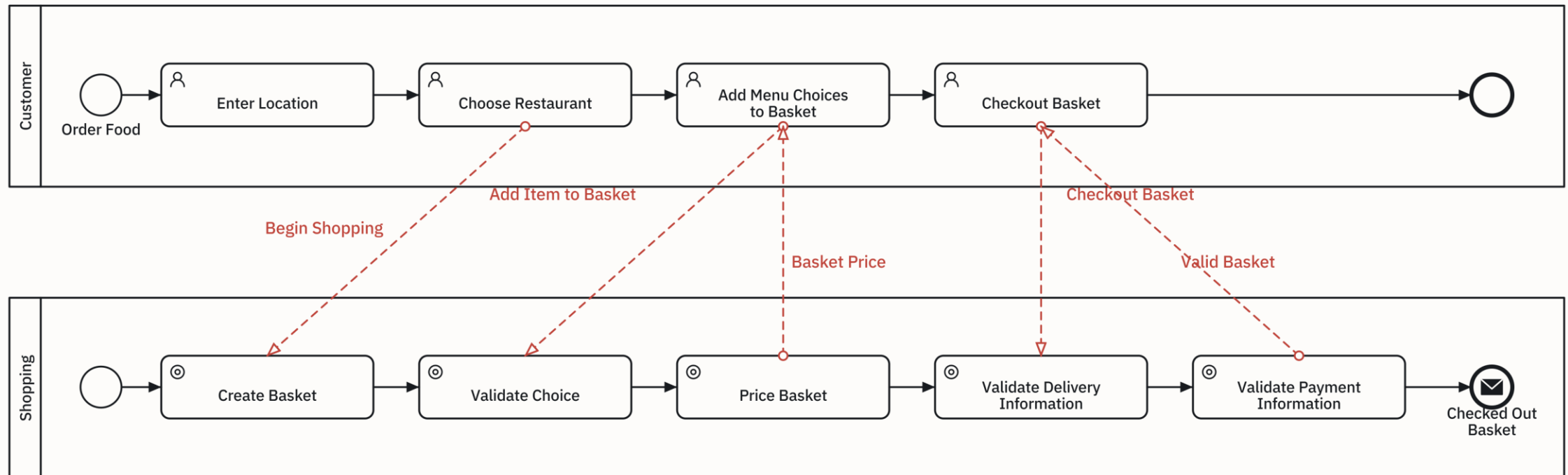
- A **Collaboration** has multiple participants, and is focused on **how they interact**.
- The message exchange between participants in a collaboration is a **Choreography** — generally the flow of messages between participants; within each orchestrated process there are message events caught or raised.
- Processes *react* to what happens in the choreography — no one owns it (it has no tokens or state of its own).

Interaction can happen two ways:

- **Within a workflow engine** — but this creates coupling, and both participants must run in the engine.
- **Via an API** — from an event-driven perspective, this is the model we care about.

Collaboration and Choreography

each pool runs its own sequence flow; the only thing crossing between them is a message

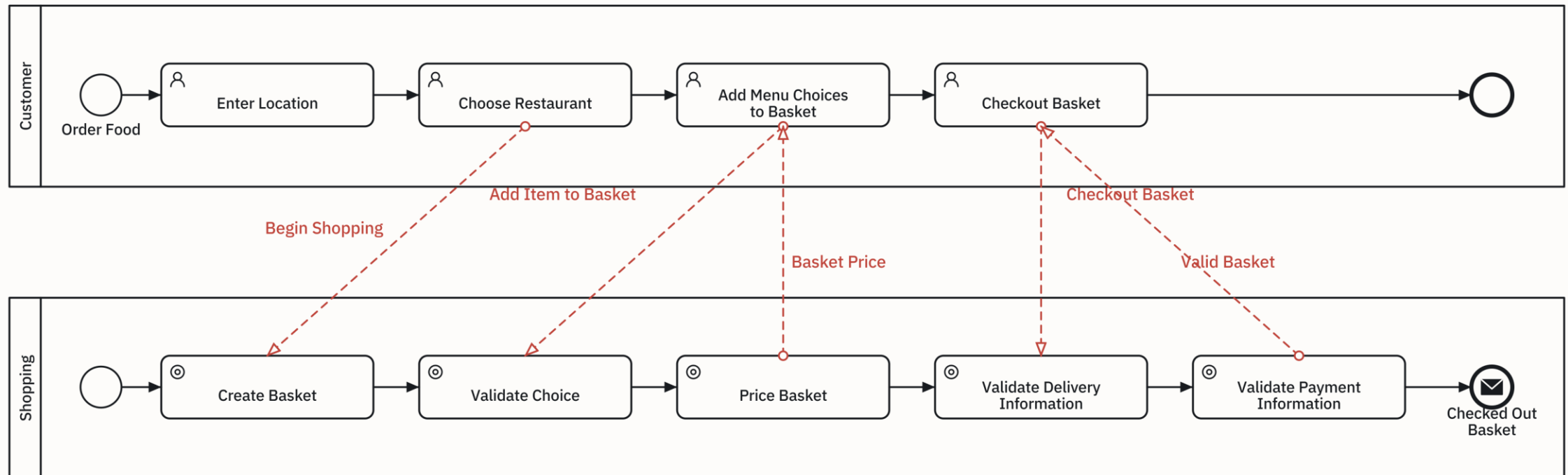


Pools and Lanes

- A **Pool** is a participant in a collaboration (a role or an organization). Not required for the main internal pool (the modeler's own org); required for other organizations' processes.
- A **Lane** distinguishes sequences of activities within a pool. White box = can see process; black box = cannot.
- Activities within the pool are organised by sequence flow.

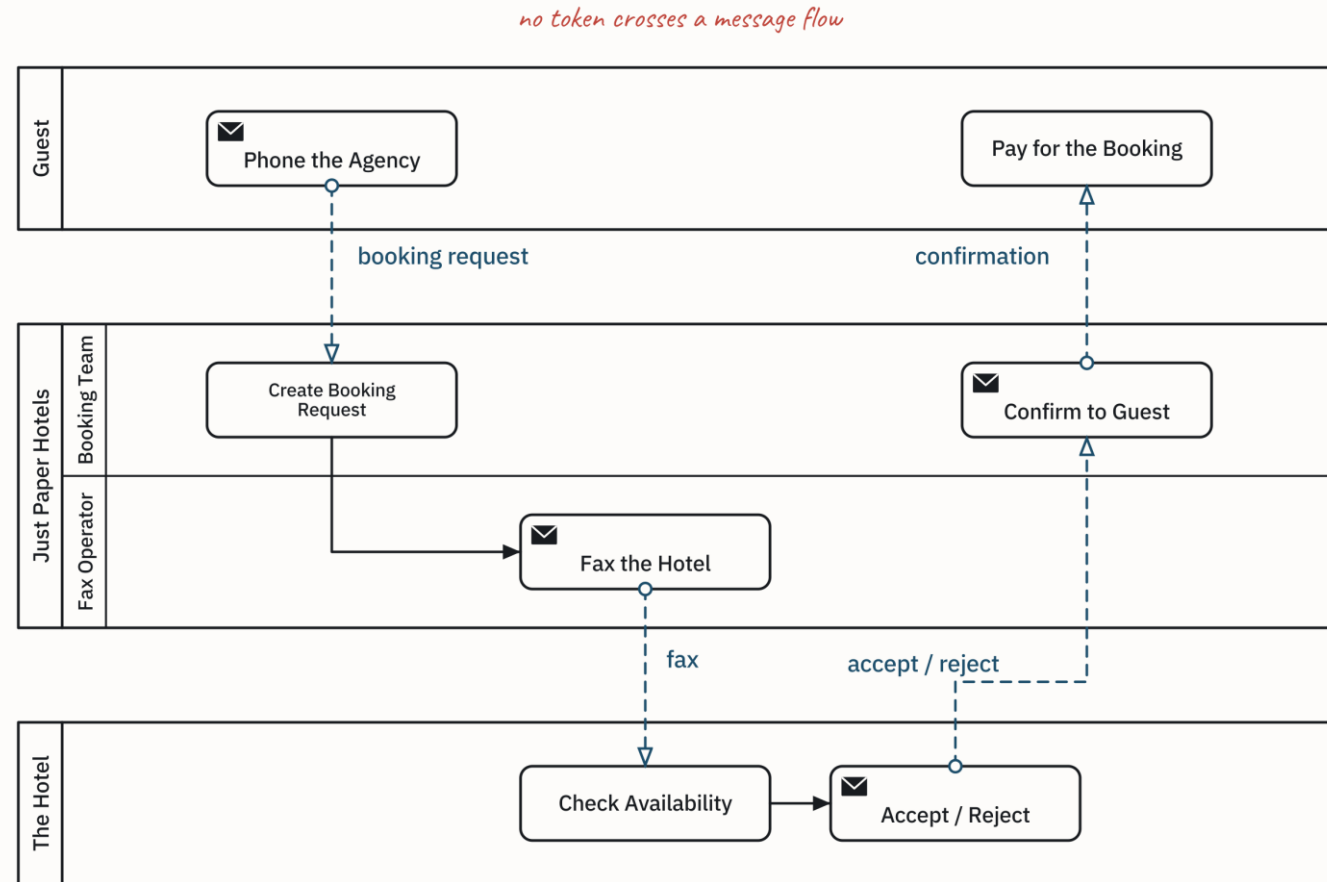
Pools and Lanes

each pool runs its own sequence flow; the only thing crossing between them is a message



Just Paper Hotels Collaboration

The full collaboration diagram for the hotel example.



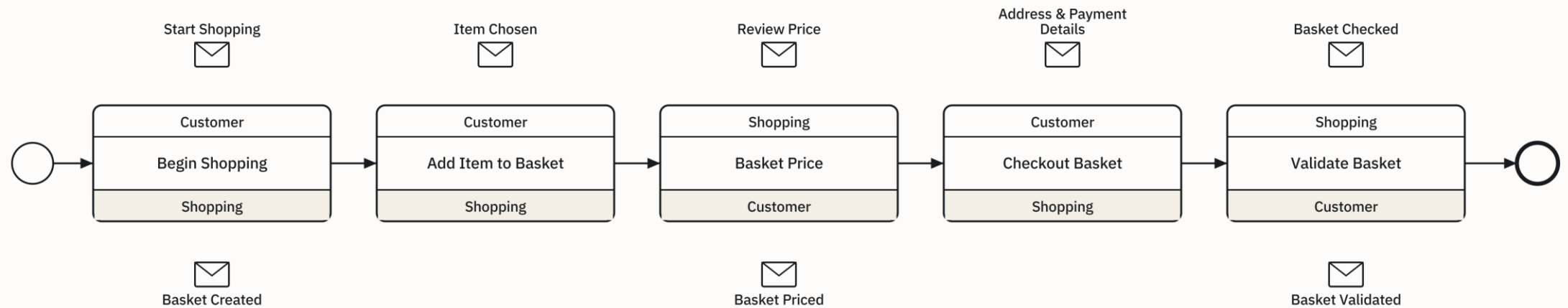
sequence flow is what happens at a desk. Message flow is what happens between desks.

Choreography and Conversation

- A **Choreography** describes a sequence/flow of activities *between* participants — a graph of flow elements (a message flow of activities, events, gateways). It is focused on the interaction across participants, **like describing a dance**: no single owner of the flow, no centralized control, and no access to anyone's shared internal state. It defines **who talks to whom, in what order**.
- A **Conversation** is a logical association of messages that can all be correlated. **Correlation Keys** associate messages in the same conversation (may be existing message data); the first task in a conversation *must* populate the conversation id.

Choreography and Conversation

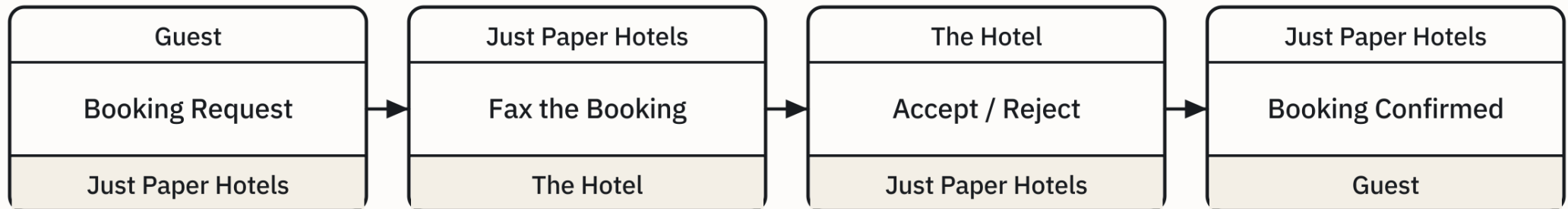
nobody owns this flow – it is the dance, not a dancer



Hotel Example – BPMN Choreography

The booking choreography – the flow of messages between participants.

no owner, no token, no shared state – only the order of the messages



the unshaded band is who speaks first. Past here the flow goes blind – the event horizon.

Misunderstandings: Choreography and Orchestration

A common EDA notion: *messaging (commands) is orchestration; eventing is choreography.*

- It is **not** a helpful rule.
- Choreography is the flow of control and information *between* orchestrations — frequently out of process, and can use messaging or eventing as required.
- An orchestration is a sequence with attendant session state (token) — frequently not distributed, within an executable process. A **routing slip** is a distributed orchestration that uses message attributes as the engine of application state.

Mistakes: Event Chaining

A sensible balance of orchestrated bounded contexts and choreography between them reduces event chaining. But **Event Pinball** can still be a problem between orchestrated services.

- **In-Out (or Robust In-Only):** explicit transfer of control reduces the "pinball effect" — creates causality by awaiting a response (send a *command* and wait for a response, not an event).
- **Out-Only:** often better at the *termination* of a workflow, or to broadcast intermediate states with no expectation of response.

Allocating Responsibility for Collaborations

- **Commands / Messaging** — transfer of control/data; typically delegating responsibility for part of a workflow. In-Out (expected to return); In-Only (not expected to return, but may on error → Robust In-Only).
- **Events** — communicate the outcome of a workflow (complete, or a milestone broadcast). Out-Only (no expected return).
- **Workflow code is glue** — glue does not execute the domain (it is not the behaviour); it *calls* something that has the behaviour.

Orchestration vs. Choreography – Summary

Aspect	Orchestration	Choreography
Control	Centralized (one participant)	Decentralized (shared)
BPMN Shape	Inside a Pool	Between Pools
Perspective	One party's process	Multi-party message exchange
Data Access	Internal	Shared messages
Execution Tool	API calls, handlers, workflow engine	Messaging protocol
Example	The agency coordinates payment and the hotel booking	Guest + Agency + Hotel interact

Durable Execution

Long-running processes must:

- Survive restarts.
- Handle retries and failures.
- Resume after waiting (e.g. for payment or the hotel's answer).

Durable execution = we **persist activity state** to indicate which steps are complete, when we're awaiting an event, etc.

Tentative Operations

A conversation that spans more than a request and a response: the requestor may not know whether the provider can succeed, and may not want to proceed without knowing.

- `reserve()` — the requestor asks if the operation is possible and reserves the resources; the provider reserves the resource (usually with a timeout) and awaits commit or rollback (`reservation()`).
- On success → `commit()` → `acknowledge()` (allocate the reserved capacity).
- On failure → `rollback()` → `freed()` (free the reserved capacity).

The booking, exactly: the agency asks the hotel to `reserve()` a room. The hotel holds it **with a timeout**, so it can sell the room to someone else if we go quiet. Guest pays inside the limit → `commit()` → `acknowledge()`, the room is allocated. Guest abandons, or the card is declined → `rollback()` → `freed()`.

Activities and Resources

Broadly, a software component manages **activities** and **resources** (Pat Helland).

- **Resources:** domain concepts we manage — hotels, rooms, guests.
- **Activities:** one or more sequences for our interaction with resources.
- As the token moves through the sequence we update resources *and* the activity (to indicate progress).
- The implementation question: **where does activity state live, and who updates it?**

Activities and Resources — Worked Example

- **Booking Team → Hotel:** `reserve()` a room → `reservation()` (waiting for response).
- **Booking Team → Payment Provider:** `take payment()` → `payment taken()` (waiting for response).
- **Activity** (usually a service task) — glue code that calls domain logic internally or messages another app; could be a framework, a bespoke state machine, or pipes and filters.
- **Resources** — code/data managing shared items coordinated across activities (room inventory, rate plans) — our domain model.
- Our workflow stores the token state (where are we in the flow?), and there is a choreography — the messages we exchange with other participants running their own flows.

Handlers + Activity State Updates

Handlers process events (change resources) and update activity state.

- Durable because state is stored (e.g. a database), but control flow is **implicit** — activity state is often implicit (e.g. within an aggregate).
- Great for simple workflows, but logic becomes scattered across handlers.
- Becomes complex with split/join/choice/merge (not a sequence), and with retry / circuit breakers / compensation.
- Relies on **guaranteed delivery** (store work for retry unless ack'd) and **Transactional Messaging (Outbox)**.

Handlers + Activity State Updates

```
var tx = await _uow.BeginOrGetTransactionAsync(ct);
try
{
    orderRepository.Received(_uow, order);           // the state change
    var canMake = await stock.Check(order);
    var reply = canMake ? new OrderAccepted(order.Id)
                       : new OrderRejected(order.Id);
    posts.Add(await _postBox.DepositPostAsync(reply, ct)); // and the message
    await tx.CommitAsync(ct);                          // both, or neither
}
catch (Exception)
{
    await tx.RollbackAsync(ct);
    throw;                                             // so it can be retried
}
await _postBox.ClearOutboxAsync(posts, ct);          // only now does it leave
```

State Machine + Activity State Updates

When handler interaction becomes complex, make the activity **explicit**.

- A **State Machine** represents the activity and triggers actions on transitions.
- Transitions are caused by receiving a message.
- The handler loads the state machine for the conversation id, triggers the transition denoted by the message, and runs the associated code.
- Save the new state and ack the message.

State Machine + Activity State Updates

```
public OrderStateMachine(ILogger<OrderStateMachine> logger)
{
    InstanceState(x => x.CurrentState);

    Initially(
        When(OnOrderSubmitted) // a message arrives...
            .Then(x => logger.LogInformation("Order submitted"))
            .SendAsync(new Uri("queue:stock-check"), Init<CheckStock>())
            .TransitionTo(PendingStock)); // ...and the state moves

    During(PendingStock,
        When(OnHasStock)
            .SendAsync(new Uri("queue:order-accepted"), Init<CallPartner>())
            .TransitionTo(AwaitingKitchen),
        When(OnLackStock)
            .TransitionTo(RejectOrder));
}
```

Routing Slip + Activity State Updates

To **distribute** steps in the sequence, transport the activity state *in the message*.

- A **routing slip** is an envelope carrying activity state (next step, context, etc.) — "Message As the Engine of Application State".
- The handler updates state when work is done, forwards according to the slip, then acks.
- Complex if "next" is dynamic routing (the code must understand how to execute).
- Complex because you distribute the steps — hard to observe (OpenTelemetry).

Workflow Engines + Activity State Updates

When we want more than state transitions (split, join, merge, choice), use a **workflow engine**.

- Explicitly models a sequence of activities and gateways; may support BPMN (or a proprietary language) with visual modelling.
- A step receives the token; the step is **glue** — invokes domain action, calls another service, sends a message.
- A **job scheduler** provides durable execution; typically any instance can resume a paused job (distributed).
- A step may wait for an event or timer to resume — the handler equivalent, or triggered by the handler calling the engine.

Compensation, Four Ways

You cannot roll back across desks, so for every "do" you write an "undo".

mechanism	where the undo lives	what triggers it	the catch
Handlers	a fallback handler per step	a fault message — In-Only needs a dedicated fault channel; In-Out marks the response as a fault	retry first, compensate only after X failures; the undo logic scatters exactly like the do logic
State Machine	a transition to a faulted state, then a compensating flow back to <i>safe</i>	the same fault message, but it drives a transition	run all fault transitions to safe <i>before</i> acking the fault message; needs durable execution, so a restart mid-fault resumes
Routing Slip	the slip's fault next — the reverse of the step just completed	context on the envelope flips to "faulted" and the slip runs backwards	the slip must carry the undo route as well as the do route
Workflow Engine	a compensation event linked to the task	retry handles technical faults; a <i>business</i> error (card declined) is a modelled branch on a gateway	with no catch, the workflow rolls back to the last wait state or terminates

| This is the **Saga**. Four costumes, one idea.

Workflow Engines — Embedded, External, and the Lessons from SOA

Embedded in a service, or an external orchestrator? The question underneath is who owns the process.

- **External orchestration is not an anti-pattern**, but it requires care.
- Keep core domain logic **inside** services; let the engine coordinate *outcomes*, not fine-grained steps.
- Consider choreography, or a local embedded orchestration, where autonomy matters more than visibility.

| "Smart endpoints and dumb pipes." — James Lewis and Martin Fowler

ESB products often include sophisticated routing, choreography, transformation and business rules. The microservice community favours the alternative, and that phrase is the name for the danger above: the more the pipe knows, the less the endpoints are allowed to.

Implementing Workflow Patterns

How the basic patterns map onto the three implementation styles:

Pattern	Handlers Only	State Machine	Workflow Engine
Sequence	Chain message handlers	State transitions (A→B→C)	Sequential steps
Parallel Split	Trigger multiple handlers	⚠️ Complex manual coordination	Parallel branches with fork/join
Synchronization	Wait for multiple states	⚠️ Hard without orchestration	Join after parallel branches
Compensation	Fallback handler	Saga	Implicit
Exclusive Choice	Conditional in handler	State transition on input	Conditional branching
Simple Merge	Trigger single follow-up	Re-entrant transition or rehydration	Multiple incoming flows, OR gateway

Putting It Together

Putting It Together

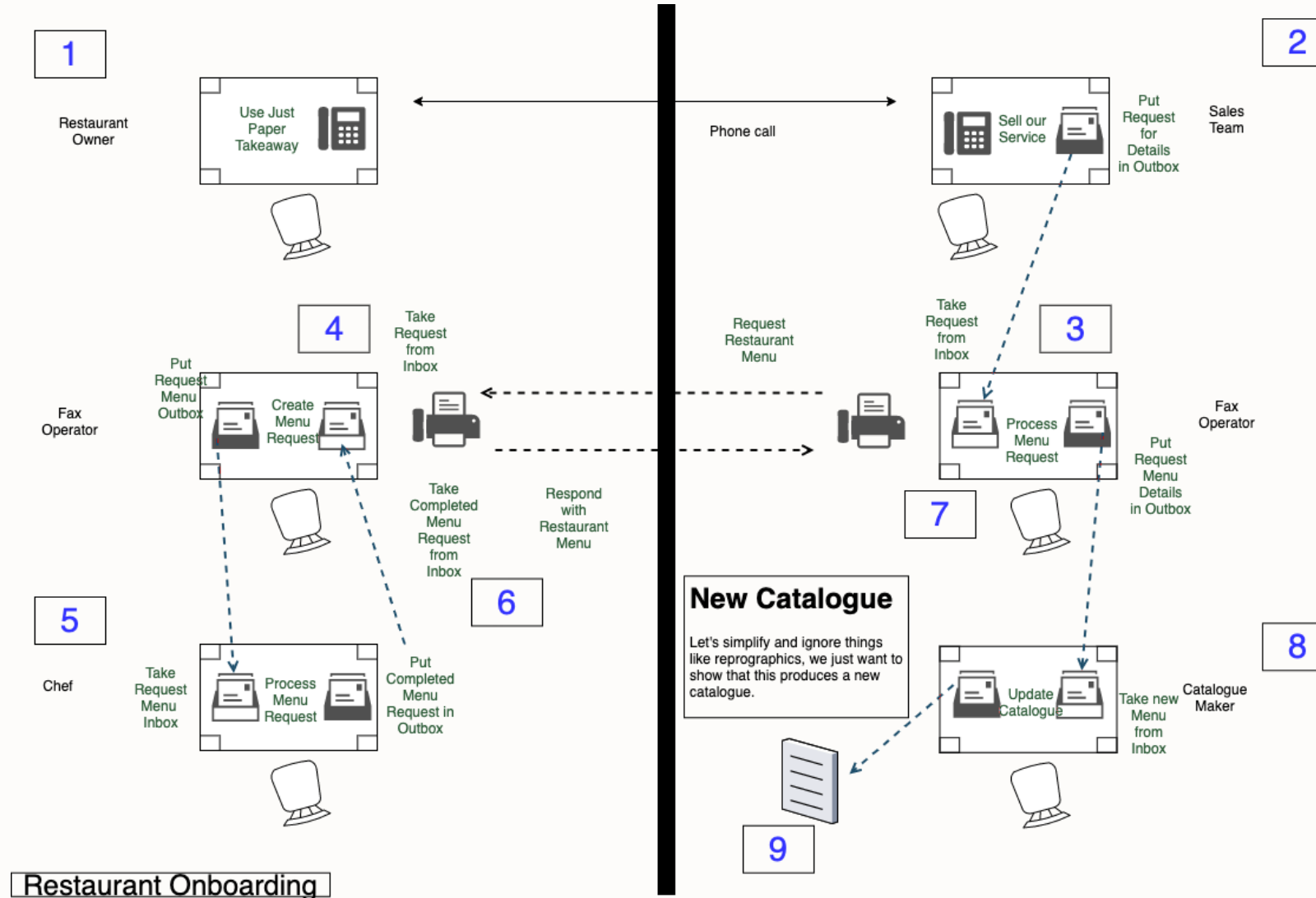
The fax workflow again — the takeaway you saw first. This time we walk it and **name every interaction as an exchange pattern**. You chose all four of them on paper this morning, before any of them had a name.

Fax Workflow — Storage and Correlation

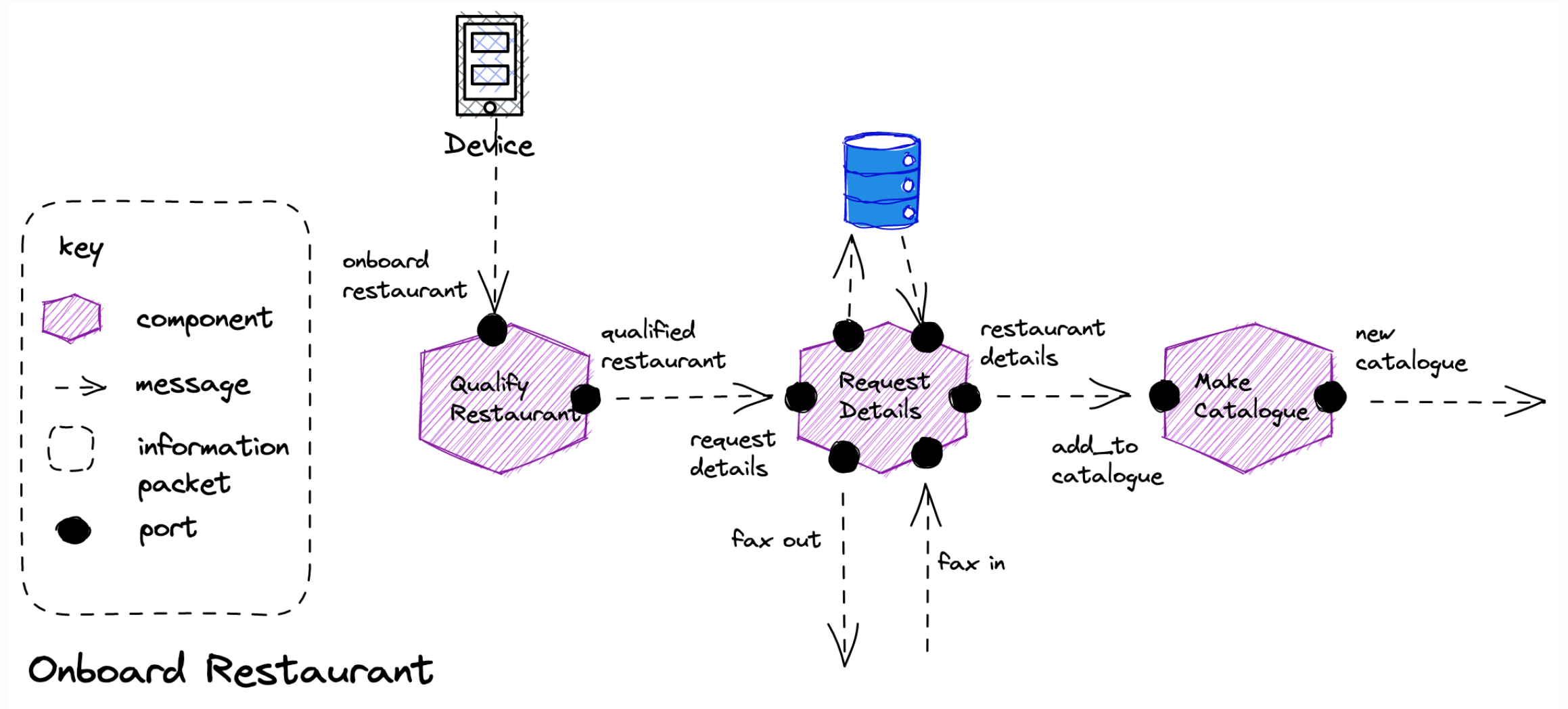
`fax_out` and `fax_in` are one **In-Out** with a gap in the middle. Every other arc on the graph is one-way — and that difference is the whole slide.

- **Storage:** store workflow state in case we crash before receiving a response.
- **Correlation:** use a correlation id sent to `fax_out` and returned via `fax_in` to look up the stored workflow.
- **Both of those are the In-Out's bill.** The one-way arcs need neither, because nothing is waiting.

Fax Workflow — Storage and Correlation



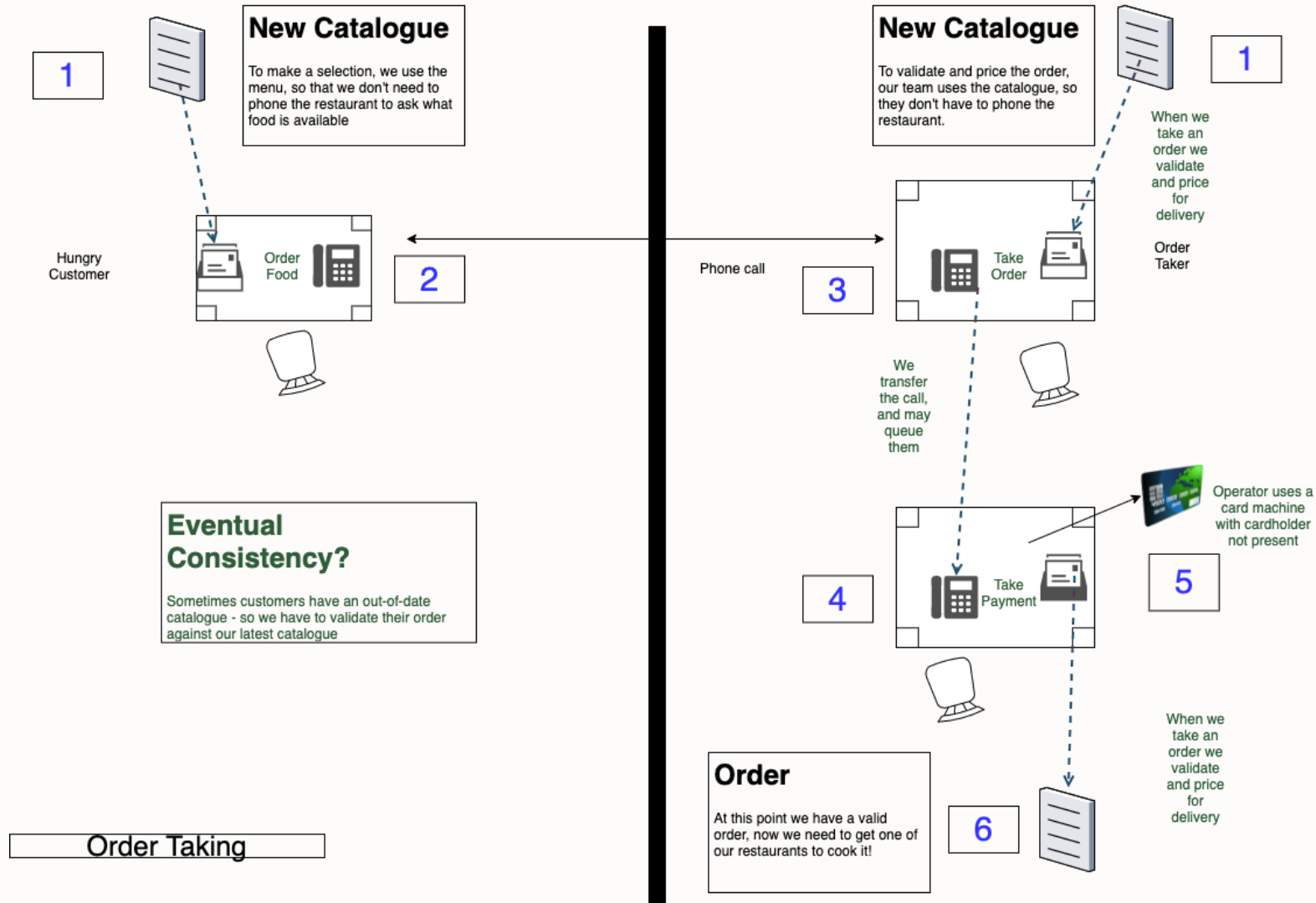
Fax Workflow — Storage and Correlation



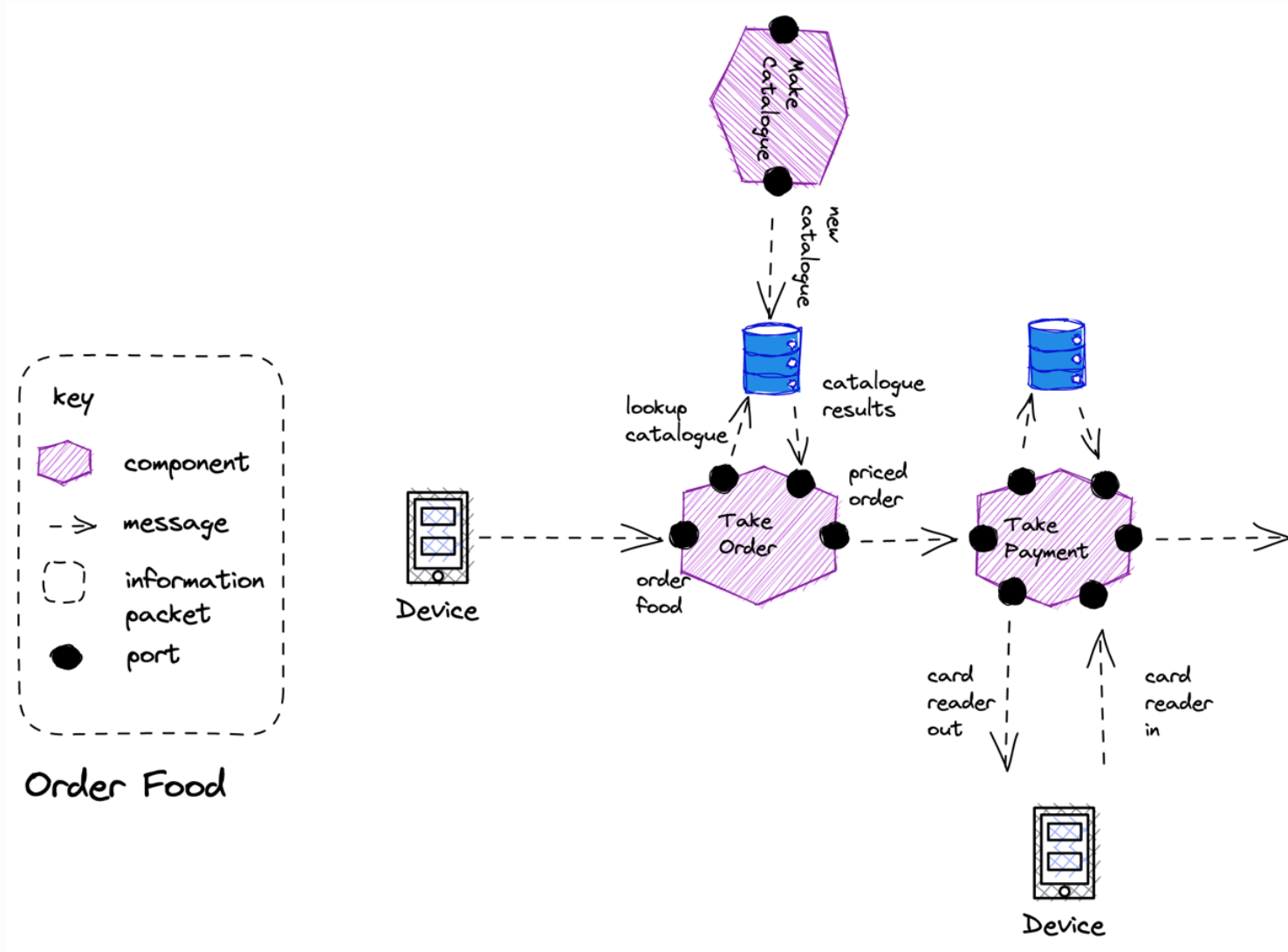
Lookup Store — the Graph Form

A component builds its lookup store from packets another component raises. **This is Day 1 §6.2 as a graph:** an Out-Only fills the store, and everything after it reads locally — so nothing in the flow makes a synchronous call to go and find the data.

Lookup Store — the Graph Form



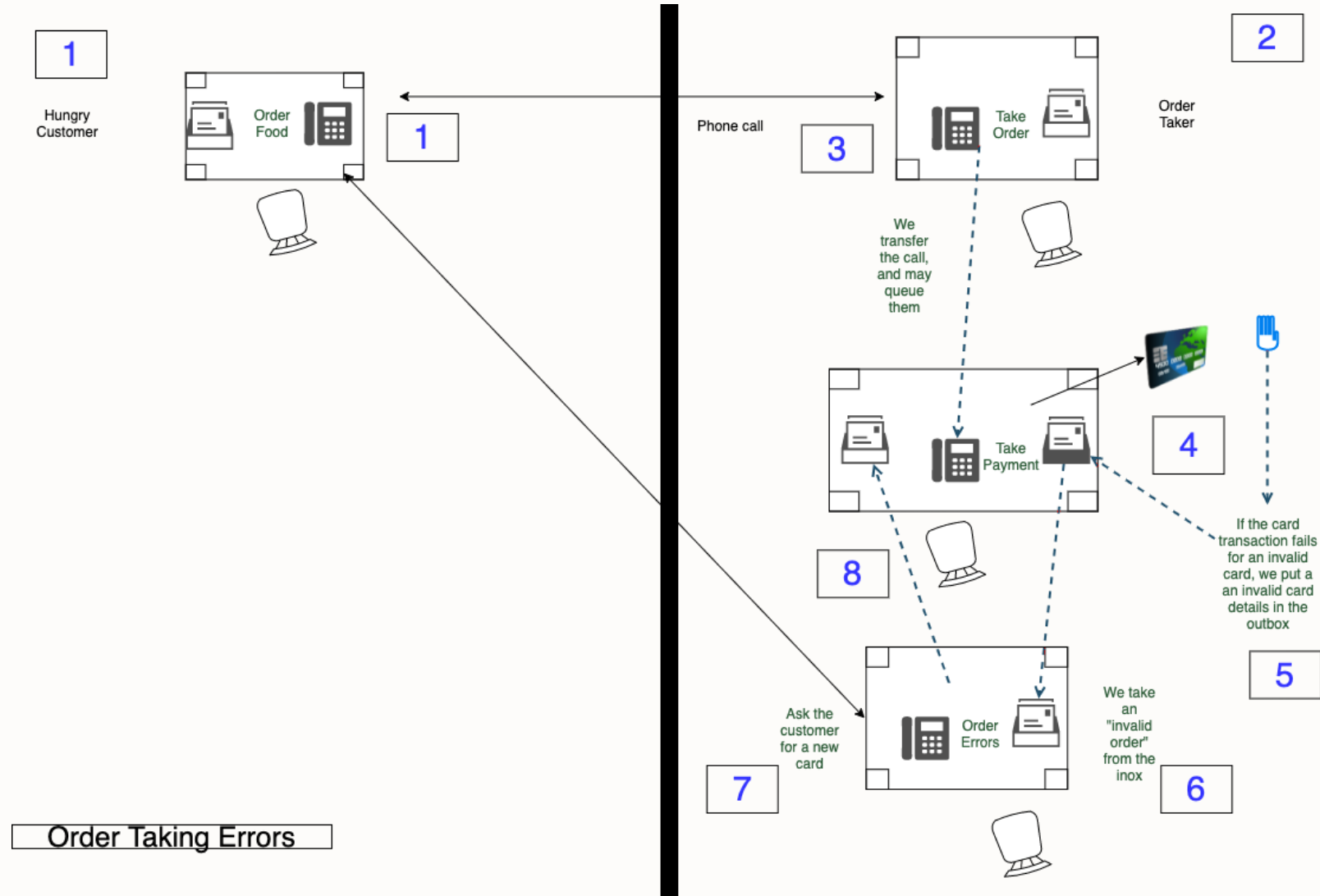
Lookup Store — the Graph Form



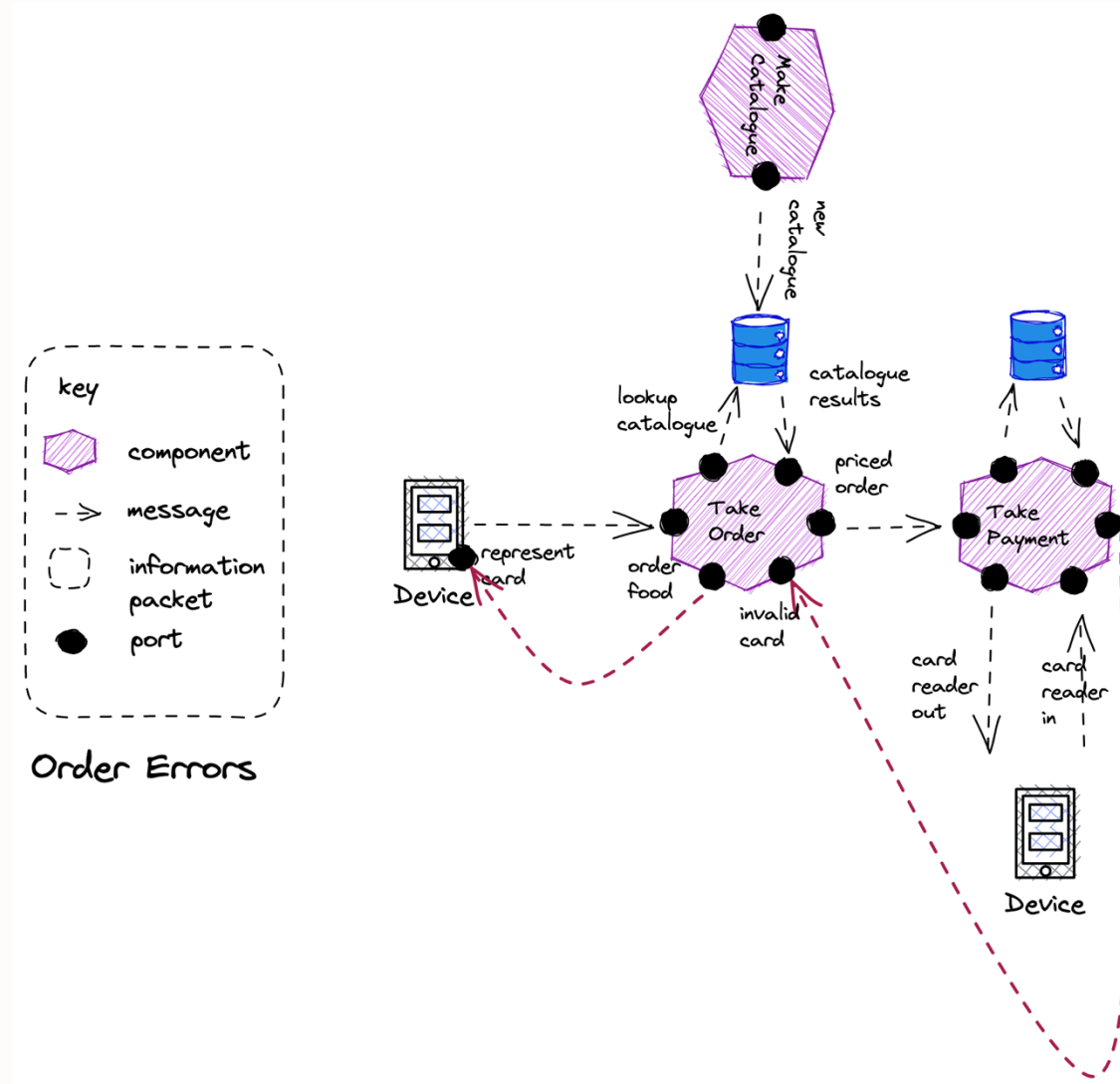
Fault Path — Message Triggers Fault

The card is declined. The fault comes back on a channel of its own — **Message Triggers Fault** — because the In-Only that started it took no response, so there is no response for a fault to replace.

Fault Path — Message Triggers Fault



Fault Path — Message Triggers Fault



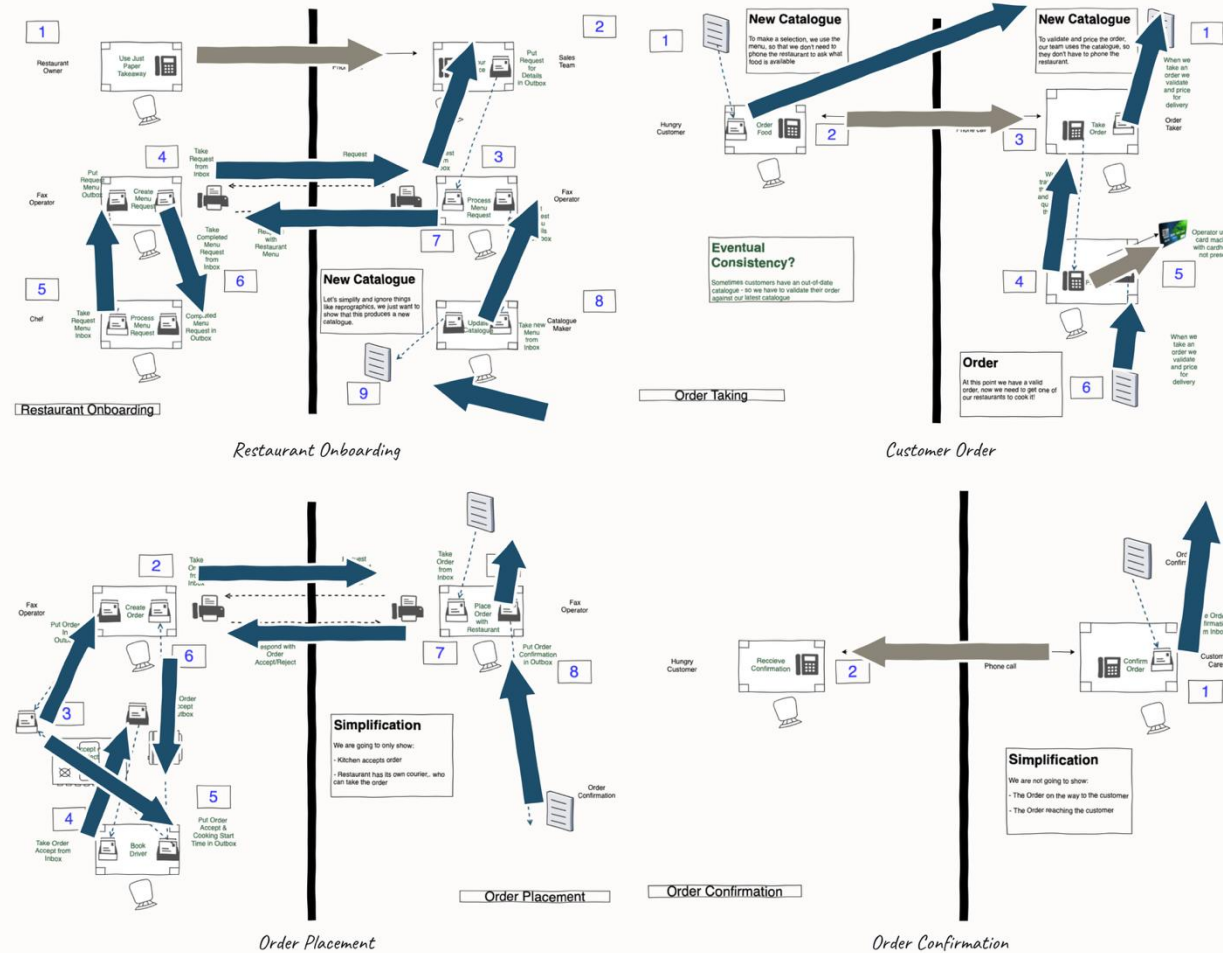
Full Flow, End to End

Everything, both ways round: the four flows as the room drew them on paper, and the whole thing as one graph. **Name each arc as you walk it, and count the In-Outs** — every one of them is a correlation id and a piece of state somebody has to own across a restart.

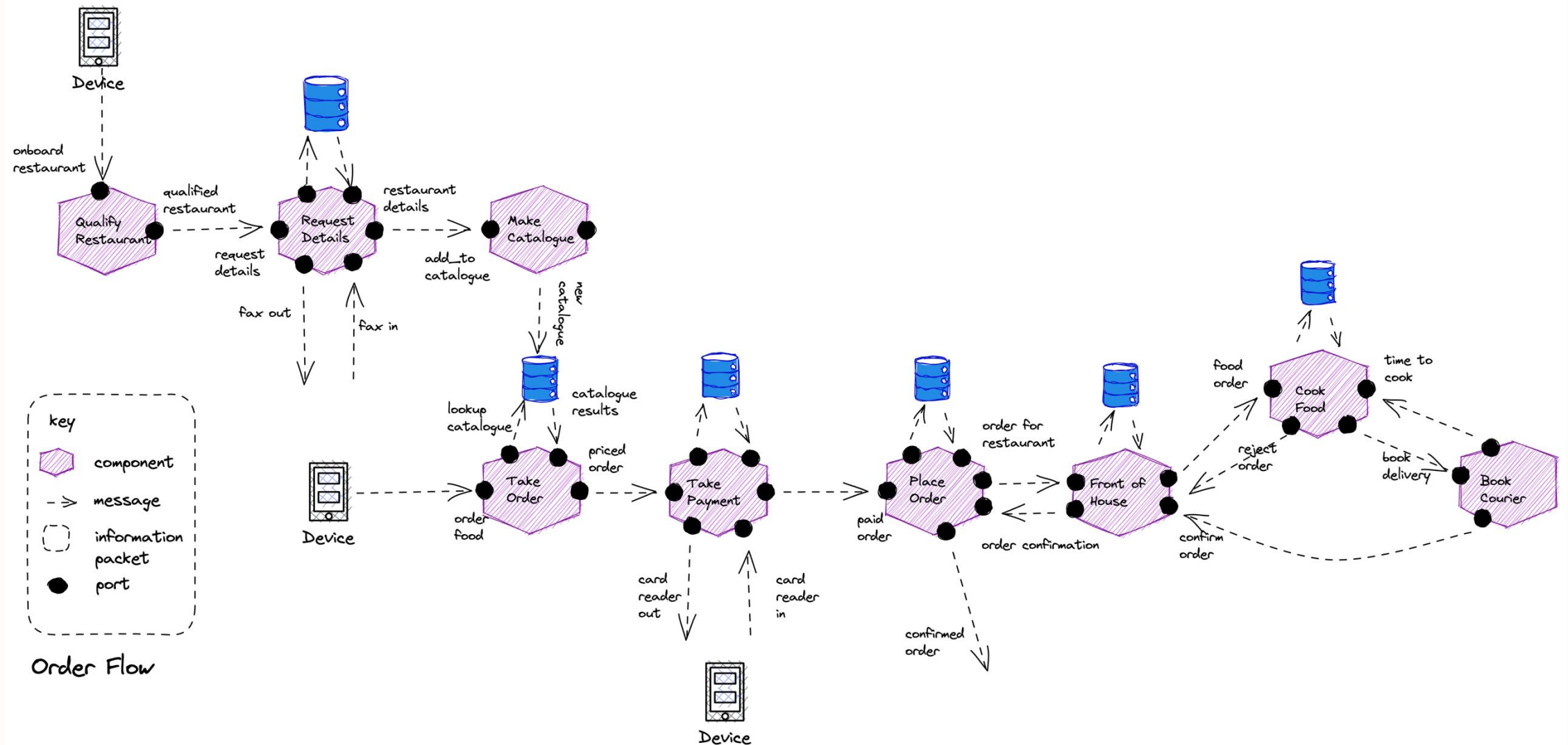
The room picked these before it knew they were choices, which is the point of having drawn them on paper first.

Full Flow, End to End

four flows, no 'main' – every desk acts because something landed in its in-tray



Full Flow, End to End



Next Steps

What we did not cover, and where it lives.

Describing and Versioning Your Messages — the Handout

Everything about the *contract* rather than the message: how you publish what your endpoints are, and how you change a message you have already published.

- **Describing endpoints** — the ABCs: Address, Binding, Contract. AsyncAPI, CloudEvents, xRegistry, and the tooling around them (EventCatalog, Backstage, AsyncAPI Studio).
- **Versioning** — Postel's Law and the **Tolerant Reader**; additive change vs. breaking change; why a breaking change means a new message type.
- **Schema and registries** — JSON Schema vs. Avro vs. Protobuf; schema registries and compatibility modes (backward / forward / full / none).

| Be strict when sending and tolerant when receiving. The rest is in the handout.

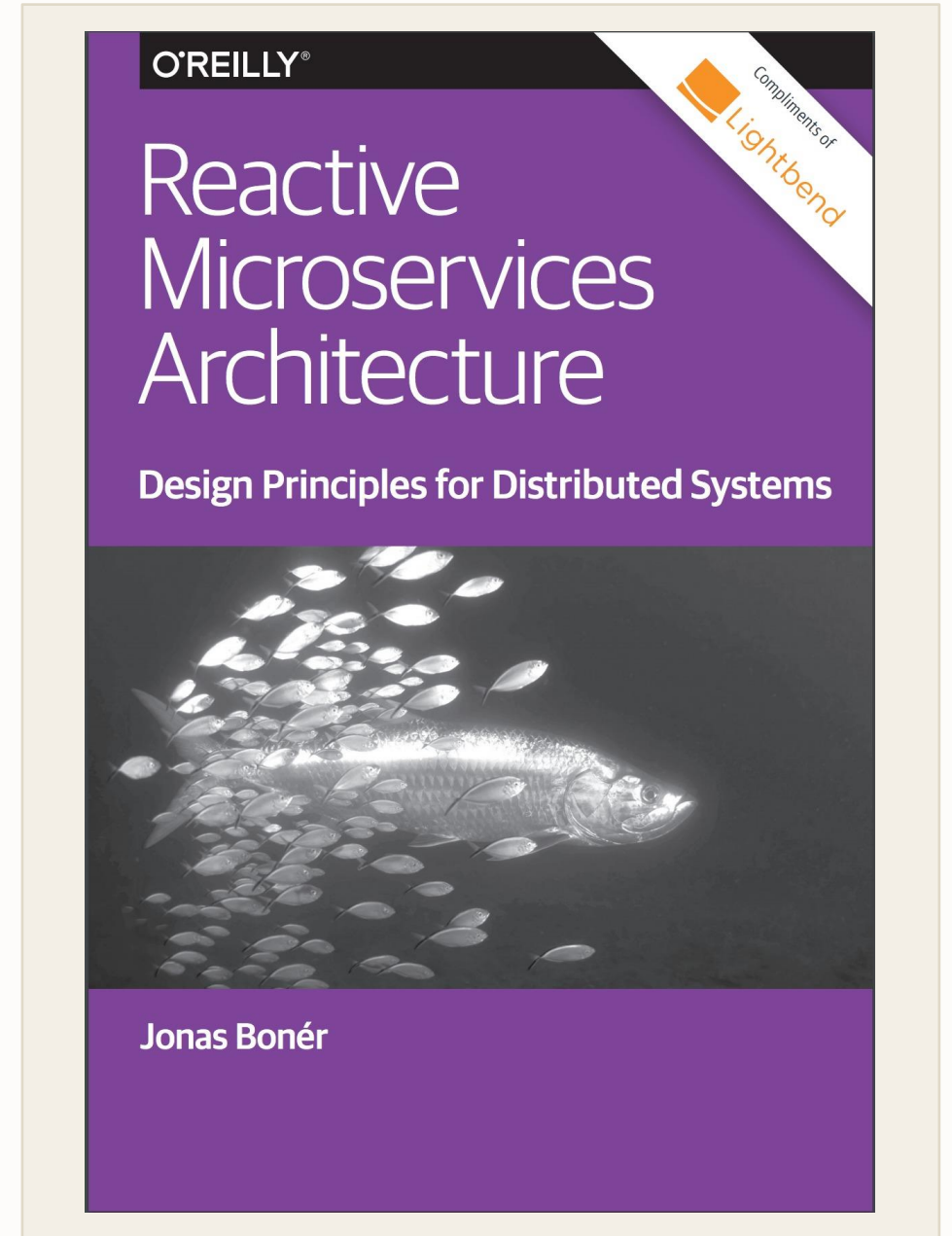
Observability

You cannot debug a flow by reading one service's logs. A message crosses process boundaries, so the trace has to cross them too.

- **OpenTelemetry** carries trace and span ids in the **message header** — the same envelope as the correlation id from the fax workflow.
- One trace spans producer, broker and every consumer, so you can see the whole conversation.
- This is the observability story for everything the two days built: the pump, the outbox, the DLQ, and the flows delegates drew on paper.

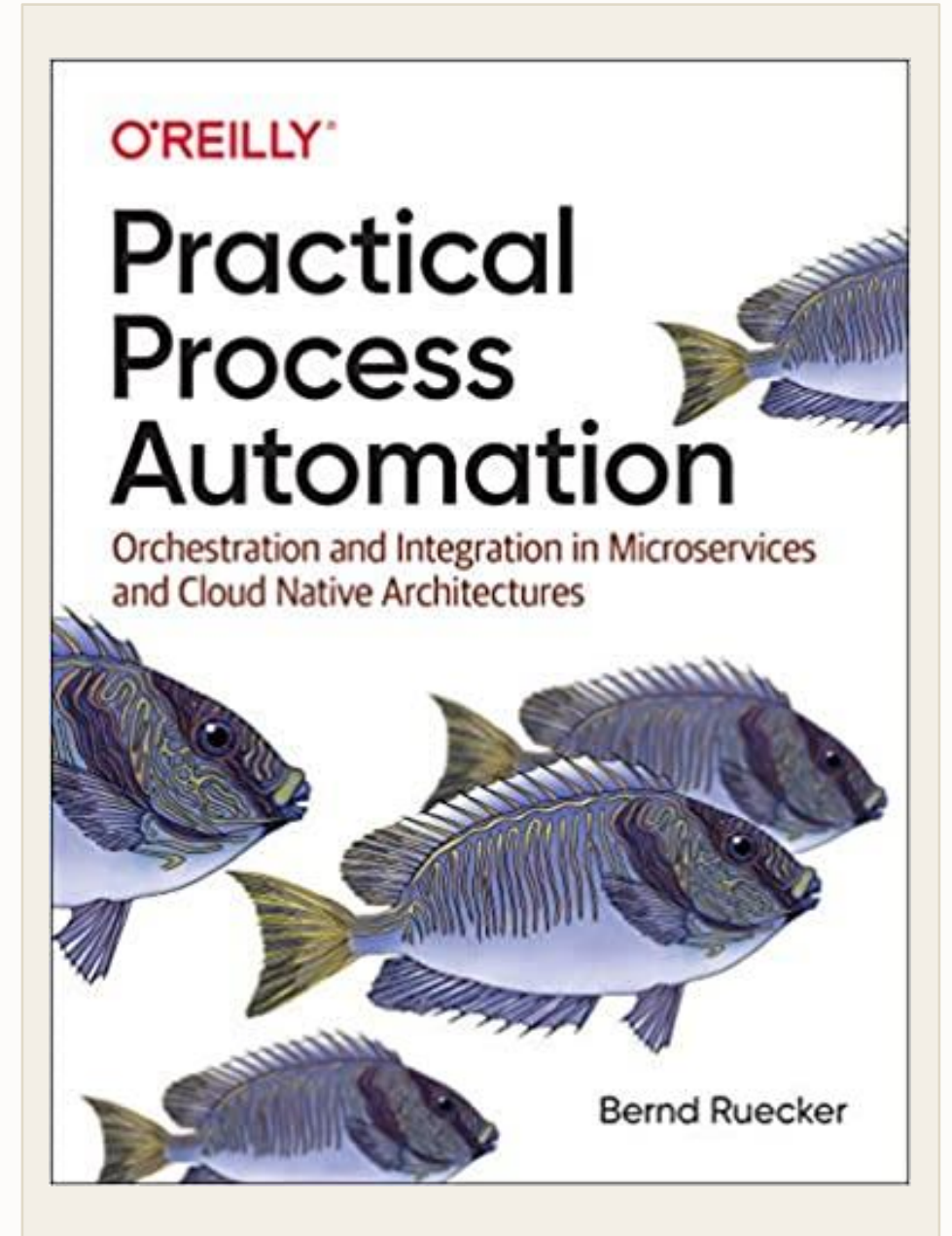
Further Reading — Reactive Microservices

Reactive Microservices Architecture — Jonas Bonér. Free PDF at jonasboner.com, under Resources.



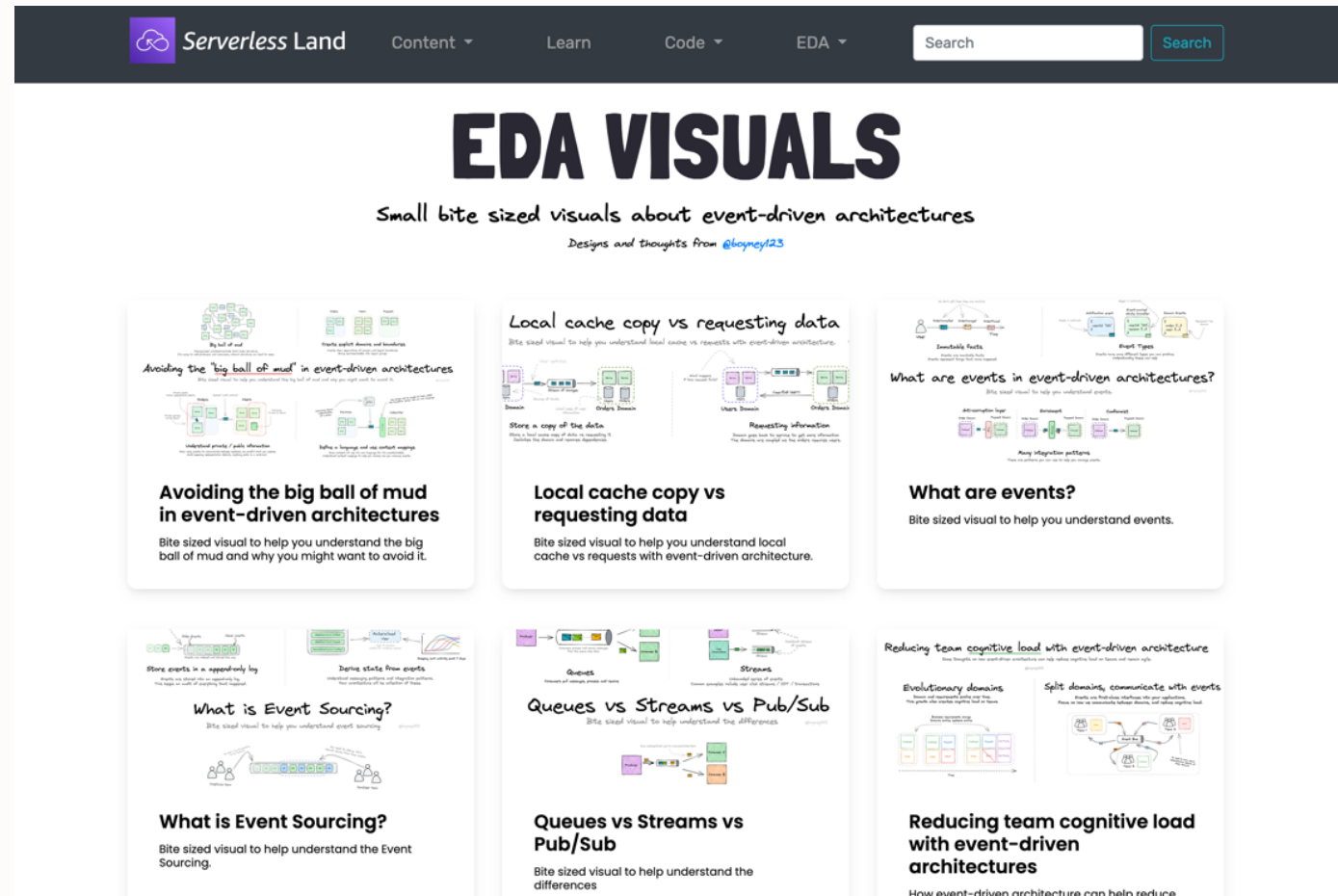
Further Reading — Practical Process Automation

Practical Process Automation — Bernd Ruecker.
(processautomationbook.com)



Further Reading — EDA Visuals

Serverless Land — event-driven architecture visuals. (serverlessland.com/event-driven-architecture/visuals)

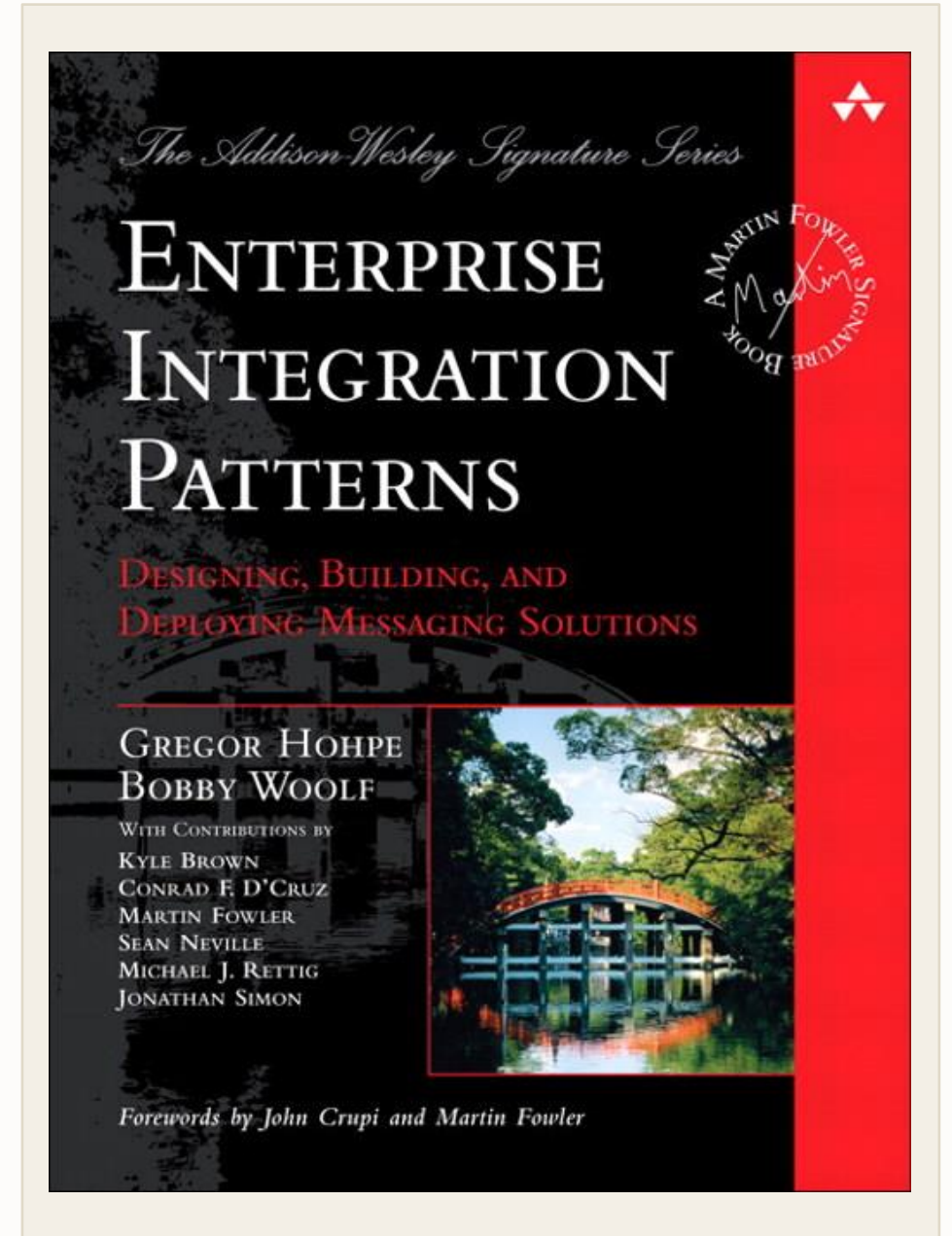


Further Reading — Enterprise Integration Patterns

Gregor Hohpe.

(enterpriseintegrationpatterns.com/gregor.html)

Eight of them are in your pack — *Routing Patterns*: Pipes and Filters, Message Translator, Content Based Router, Dynamic Router, Recipient List, Splitter, Aggregator, Resequencer.



NEXT STEPS

Q&A

Questions and discussion.